MANCHESTER 1824
The University of Manchester

AUTOMATED DETECTION, TRACKING AND TACTICAL ANALYSIS IN BROADCAST FOOTBALL VIDEO

A REPORT SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF BACHELOR OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

Aryan Tiwari

Supervised by Dr Aphrodite Galata

Department of Computer Science

Contents

Abstract **9**

Declaration **10**

Copyright **11**

Acknowledgements **12**

1 Introduction **13**
1.1 Motivation and Context 13
1.2 Aims and Objectives 14
1.3 Evaluation Strategy 15
1.4 Report Structure 16

2 Background and Theory **18**
2.1 Broadcast Football Video Analysis 18
2.2 Related Work 19
2.2.1 Commercial Tracking Systems 19
2.2.2 Academic Benchmarks and Integrated Pipelines 19
2.2.3 Positioning of This Project 20
2.3 Machine Learning and Computer Vision Foundations 20
2.3.1 Supervised Learning for Visual Recognition 21
2.3.2 Neural Networks 21
2.3.3 Convolutional Neural Networks 22
2.3.4 Transfer Learning and Fine-Tuning 23
2.4 Object Detection in Broadcast Football 23
2.4.1 Bounding Boxes, IoU, and Non-Maximum Suppression . . . 24
2.4.2 Precision, Recall, and Mean Average Precision 25

2.4.3 The YOLO Architecture 26
2.5 Multi-Object Tracking 28
2.5.1 Tracking-by-Detection 29
2.5.2 Motion Prediction and Data Association 29
2.5.3 ByteTrack 30
2.5.4 BoT-SORT 32
2.5.5 Tracking Evaluation: HOTA 32
2.6 Team Identification from Visual Features 32
2.6.1 Team Identification as Clustering 33
2.6.2 Visual Representations 33
2.6.3 Dimensionality Reduction 34
2.6.4 k-Means Clustering 35
2.7 Pitch Registration and Planar Homography 35
2.7.1 Planar Homography 36
2.7.2 Estimating H from Point Correspondences 36
2.7.3 Pitch Keypoint Detection 37
2.7.4 Reprojection Error 38
2.8 Spatial and Tactical Analysis in Pitch Space 38
2.8.1 Pitch-Space Trajectories 38
2.8.2 Heatmap Construction 39
2.8.3 Speed and Distance Estimation 39
2.8.4 Possession via Nearest-Player Assignment 40
2.8.5 Offside Rule Formalisation 41
2.9 Chapter Summary 41

3 Methodology and Implementation **43**
3.1 System Overview 43
3.2 Requirements and Design Objectives 46
3.3 Development Methodology 46
3.4 Object and Ball Detection 48
3.4.1 Generic Four-Class Detector 48
3.4.2 Dedicated Ball Detector 49
3.5 Multi-Object Tracking 50
3.6 Team Classification 51
3.6.1 Warmup-Phase Integration 51
3.6.2 Torso Cropping 52

3.6.3 Track-Level Temporal Smoothing . . . 52
3.6.4 Goalkeeper Resolution . . . 53
3.7 Pitch Registration . . . 54
3.7.1 Keypoint Detector . . . 54
3.7.2 Stride-Cached Homography Estimation . . . 55
3.8 Tactical Reasoning . . . 56
3.8.1 Pitch-Space Projection . . . 56
3.8.2 Possession Tracking . . . 56
3.8.3 Pass Detection . . . 57
3.8.4 Offside Detection . . . 57
3.8.5 Speed and Distance . . . 58
3.8.6 Heatmaps . . . 58
3.9 Implementation Testing and Verification . . . 59
3.10 System Engineering and Reproducibility . . . 59
3.10.1 Pipeline Orchestration . . . 59
3.10.2 Output Formats . . . 61
3.10.3 Reproducibility . . . 61
3.11 Design Decisions and Failure Handling . . . 62

4 Evaluation **65**
4.1 Analysis . . . 65
4.2 Discussion . . . 65
4.3 Limitations . . . 65

5 Conclusion **66**
5.1 Summary . . . 66
5.2 Future Work . . . 66

A Additional Materials **72**

Word Count: 14700 (TODO: update at the end)

List of Tables

1.1 Summary of evaluation metrics used in this project. 16

3.1 Intermediate representations passed between pipeline components, with the section defining each producer indicated in parentheses. 45
3.2 Functional (FR) and non-functional (NFR) requirements, with the originating Chapter 1 objective (Obj.) and success criterion (SC) where applicable, and the implementation section that delivers each requirement. 47
3.3 Ball-class performance on the held-out test set: generic four-class detector versus dedicated single-class detector. The dedicated detector raises ball mAP@50 from 0.459 to 0.950 — a 49 percentage-point absolute improvement that motivates the two-detector architecture. Per-class metrics for all classes of the generic detector are reported in Chapter 4. 50
3.4 Component-level verification performed during development. Each row describes a deliberate inspection step performed before the corresponding component was relied on in the integrated pipeline. Quantitative evaluation is reported separately in Chapter 4. 60
3.5 Principal architectural decisions, their considered alternatives, and the trade-offs each decision accepts. 63
3.6 Failure conditions handled by the pipeline, the corresponding mitigation strategy, and the residual limitation that the mitigation does not fully address. The half-time direction-switch row resolves the forward reference made in Section 3.8.4. 64

List of Figures

2.1 A common CNN architecture: stacked convolution, activation, and pooling stages produce feature maps of decreasing spatial resolution, which are passed to fully connected layers for prediction. Reproduced from O'Shea and Nash [32]. 23

2.2 Intersection over Union (IoU) between a ground-truth bounding box B_g (blue, dashed) and a predicted bounding box B_p (orange). IoU is the ratio of the intersection area (gold) to the total union area of both boxes, computed as $\text{IoU} = |B_g \cap B_p| / |B_g \cup B_p|$. 24

2.3 Illustration of Non-Maximum Suppression. Before NMS, multiple overlapping candidate boxes may be produced for the same object. After NMS, lower-confidence overlapping boxes are suppressed and the most confident detection is retained. 25

2.4 Generalised YOLO-style single-stage detection pipeline. The backbone extracts multi-scale feature maps, the neck fuses them, and the detection head predicts boxes, objectness, and class scores. NMS removes duplicate predictions to produce the final detection set $\{(b_i, c_i, s_i)\}$. 27

2.5 ByteTrack two-stage association process. Detector output is partitioned by confidence threshold τ_{high} into a high-score set D_t^{high} and a low-score set D_t^{low}. Stage 1 matches active tracks against D_t^{high} using the Hungarian algorithm with IoU cost; unmatched tracks proceed to Stage 2, where they are compared against D_t^{low}. Tracks that remain unmatched after both stages are held in a buffer and deleted if unrecovered within a fixed number of frames. 31

3.1 Pipeline architecture. Three parallel branches process each frame: a generic object detector with multi-object tracker producing per-frame player and referee detections that are then assigned to teams (left); a dedicated ball detector with a temporal coherence filter (centre); and a pitch keypoint detector whose outputs are used to estimate a per-frame homography by Direct Linear Transform (right). The three branches converge at a shared pitch-space representation, on which all spatial analytics — heatmaps, speed and distance, possession, passes, and offside — are computed. 44

List of Abbreviations

HOTA	Higher Order Tracking Accuracy
IDF1	Identification F1
IoU	Intersection over Union
MAE	Mean Absolute Error
mAP	mean Average Precision
MOTA	Multi-Object Tracking Accuracy

Abstract

TODO: my abstract (to be done at the end of the report)

Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Aphrodite Galata, for her invaluable guidance, support, and advice throughout this project.

I am also grateful to Kunj Dhola and Selim Sayed for their collaboration and contributions to the project.

I acknowledge the use of generative AI tools to improve the written English, spelling, and grammar of text that I had drafted myself, in line with the project's permitted use of AI. The technical content, methodology, system design, experimental work, analysis, and conclusions presented in this report are my own. A full declaration of AI usage is provided in the project's AI declaration form submitted alongside this report.

Finally, I would like to thank my family and friends for their continued encouragement and support throughout my studies.

Chapter 1

Introduction

1.1 Motivation and Context

The use of data and analytics in professional football has grown substantially in recent years, with clubs and national federations increasingly incorporating analytical methods into performance analysis and decision-making [25]. In particular, positional and tracking data are valuable because they provide richer descriptions of player movement, team organisation, and tactical behaviour than traditional notational analysis alone [11, 42]. As a result, access to reliable spatial match data has become increasingly important.

Despite this, high-quality football data remain difficult to access outside elite settings. Advanced event and tracking data are often proprietary, expensive, or dependent on specialist collection systems, which limits their availability for public and reproducible work [22, 9, 6]. This motivates the investigation of alternative ways to generate structured football data from more widely available sources.

Broadcast video is an attractive candidate because it is widely available and already contains much of the spatial information needed for tactical analysis. However, extracting reliable structured data from broadcast footage is challenging, since it requires several interconnected tasks such as player detection, tracking, team assignment, and pitch registration under conditions including camera motion, occlusion, scale variation, and motion blur [23, 3, 41, 40].

Recent research has shown progress on many of these components and, increasingly, on parts of the broader workflow. Open benchmark efforts such as SoccerNet-Tracking and SoccerNet Game State Reconstruction demonstrate that broadcast football video can support tracking and game-state reconstruction in realistic settings [5,

38]. However, these systems typically produce intermediate outputs such as tracked player positions or minimap reconstructions rather than directly usable tactical outputs. What remains missing is a publicly reproducible end-to-end system that integrates all major stages from broadcast video to tactical output using modern computer vision methods.

1.2 Aims and Objectives

To address this gap, the project investigates how far a modular computer vision pipeline can extract reliable tactical and spatial information from broadcast football video without requiring specialised capture systems or manual team labels.

To achieve this aim, the project pursues the following objectives:

- develop and evaluate a detection approach for the main on-pitch entity classes—players, goalkeepers, referees, and the ball—from broadcast football footage;
- assess whether multi-object tracking can preserve sufficiently consistent identities over time to support downstream tactical analysis;
- test whether unsupervised clustering of visual embeddings can separate players by team without manual team labels;
- measure the geometric accuracy of pitch keypoint detection and homography estimation for projecting image-space positions into pitch-space;
- determine whether the resulting projected trajectories can support higher-level tactical outputs, including bird's-eye visualisations, heatmaps, possession estimates, pass events, and offside-related analysis; and
- evaluate the pipeline as an end-to-end system, including how errors in detection, tracking, and homography estimation affect downstream analytical outputs.

The project does not aim to replace professional commercial tracking systems, but rather to assess how much tactical information a reproducible, open pipeline can extract from standard broadcast footage.

The project will be considered successful if: (i) the main object detector achieves $\text{mAP}_{50} \geq 0.80$ across entity classes; (ii) multi-object tracking reaches HOTA ≥ 0.45 on a labelled test segment; (iii) unsupervised team assignment achieves $F_1 \geq 0.85$; and

(iv) the integrated pipeline produces coherent tactical outputs—including bird's-eye projections, heatmaps, and offside analysis—that are qualitatively consistent with the underlying match footage. These thresholds reflect realistic performance expectations for broadcast football analysis informed by the literature [41, 5].

1.3 Evaluation Strategy

This project will be evaluated at both *component level* and *system level*. This is necessary because downstream outputs such as possession estimates, heatmaps, and offside decisions depend not only on the performance of individual modules, but also on how errors propagate across detection, tracking, and pitch registration stages [41, 40]. The evaluation therefore asks two questions: whether each component performs adequately on its own task, and whether the integrated pipeline produces coherent and tactically meaningful outputs as a complete system.

At component level, each module will be assessed using standard task-appropriate metrics from the computer vision literature. At system level, the pipeline will be judged by the coherence, plausibility, and usefulness of its final outputs on representative broadcast sequences. The chosen metrics reflect the main technical risks of the project: detection accuracy, temporal identity consistency, geometric projection quality, and the validity of downstream tactical inference. These metrics were chosen because each targets a distinct failure mode of the pipeline. Mean Average Precision (mAP) is the standard object detection metric in the computer vision literature and directly measures whether the detector can reliably localise and classify entities under broadcast conditions [41]. Higher Order Tracking Accuracy (HOTA) was selected over legacy Multi-Object Tracking (MOT) metrics such as Multi-Object Tracking Accuracy (MOTA) because it balances detection and association quality in a single score, making it better suited to evaluating identity consistency over time [26]. Reprojection error measures how accurately estimated homographies map known pitch points, which determines whether projected player positions are geometrically meaningful.

At system level, quantitative metrics alone cannot capture whether tactical outputs are useful, so qualitative assessment against the source footage is included following the approach used in recent game-state reconstruction benchmarks [38]. The numeric thresholds were set based on reported performance in comparable broadcast football settings: for example, Theiner et al. [41] report mAP_{50} values in the range 0.80–0.90 for player detection, and SoccerNet-Tracking baselines report HOTA scores

in the 0.40–0.55 range [5], making the chosen targets realistic but non-trivial. Detailed annotation procedures and experimental methodology are presented in Chapter 4.

Table 1.1: Summary of evaluation metrics used in this project.

Component / Output	**Primary metrics**	**Purpose**
Object detection	$mAP_{50–95}$, mAP_{50}, precision, recall	Measure localisation and classification performance for players, goalkeepers, referees, and the ball.
Ball detection	mAP_{50}, precision, recall, false positive rate	Assess robustness under small object size, motion blur, and occlusion.
Pitch keypoints and homography	$mAP_{50–95}$ for keypoints; reprojection error for homography	Measure field registration quality and projection accuracy in pitch-space.
Multi-object tracking	HOTA, IDF1, MOTA, identity switches	Evaluate temporal consistency and identity preservation [26, 8].
Team assignment	Precision, recall, F_1 score	Measure accuracy of unsupervised team separation.
Tactical outputs	Precision, recall, and F_1 for event detection; MAE for possession; qualitative assessment for heatmaps and bird's-eye views	Assess whether projected trajectories support useful downstream analysis.
End-to-end pipeline	Integrated case study and failure analysis	Evaluate output coherence and identify how upstream errors affect downstream reasoning.

Component-level evaluation shows whether each stage meets its technical objective, while system-level evaluation tests whether the combined pipeline remains analytically useful once these stages are linked. The final judgement of success will therefore be based on both quantitative component metrics and qualitative assessment of whether the integrated outputs remain consistent with the source footage and useful for tactical interpretation.

1.4 Report Structure

The remainder of this report is organised to guide the reader from the background knowledge needed to understand the system, through its design and outputs, to a critical assessment of its performance. Chapter 2 (*Background and Theory*) reviews the relevant literature and technical foundations, establishing the context from which the pipeline design decisions follow. Chapter 3 (*Methodology and Implementation*) describes the architecture and implementation of each pipeline stage, building directly on

the methods and models introduced in the background. Chapter **??** (*Results*) presents the outputs produced by the system in practice. Chapter 4 (*Evaluation*) then assesses the project at both component and system level, using the metrics and strategy outlined in Section 1.3, including failure analysis and the effect of error propagation on downstream outputs. Finally, Chapter 5 (*Conclusion*) summarises the main achievements, reflects on limitations, and outlines directions for future work.

Chapter 2

Background and Theory

2.1 Broadcast Football Video Analysis

Broadcast football footage poses several coupled technical challenges that distinguish it from analysis based on dedicated tracking infrastructure [41, 23]. Where multi-camera or sensor-based systems can rely on fixed reference frames and direct measurement, a broadcast pipeline must recover all player, ball, and pitch information from a single moving-camera video stream [11].

Four properties of this medium drive the design of any such pipeline. The camera continuously pans, tilts, and zooms, so the background cannot serve as a fixed spatial reference and inter-frame motion cannot be attributed to scene objects alone [41]. The ball is small relative to the frame and is frequently blurred, occluded, or temporarily absent, making its detection substantially harder than that of players [5]. Players on the same team share near-identical appearance — uniform colour, similar build — so identity must be inferred from subtle spatial and temporal cues [41]. Finally, image-space measurements suffer perspective distortion, so quantities such as distance, speed, and tactical position are meaningful only after registration to a canonical pitch model [40].

These constraints have driven the field toward a modular pipeline [41, 23]: object and ball detection produce per-frame observations; multi-object tracking maintains temporal identity; team assignment recovers affiliation; pitch registration projects coordinates into field space; and higher-level outputs — heatmaps, possession, offside — are computed in pitch coordinates rather than in the image plane.

Formally, the pipeline is a composition of stages,

$$y = f_n(f_{n-1}(\cdots f_1(x))), \tag{2.1}$$

where x is the input video, each f_k is a processing stage, and y is a final pitch-space output such as a heatmap, possession estimate, or offside decision. This composed structure has an important consequence: every output depends on the quality of every upstream stage, so the pipeline cannot be analysed by considering its components in isolation.

2.2 Related Work

This section reviews existing commercial systems, public benchmarks, and academic pipelines for broadcast football analysis, in order to identify the gap that motivates an open pipeline producing tactical outputs from broadcast video alone.

2.2.1 Commercial Tracking Systems

Professional clubs typically obtain positional data from commercial optical or sensor-based systems. TRACAB uses multi-camera optical tracking and has been shown to attain sub-metre positional accuracy under validated match conditions [22], while providers such as Hudl StatsBomb and Second Spectrum supply event data, tracking data, and tactical analysis tools to clubs and broadcasters [6]. These systems are not, however, a viable foundation for open research: their algorithms, data-collection procedures, and raw outputs are proprietary, which limits reproducibility, fair comparison across methods, and applicability to matches where commercial tracking data are unavailable [9, 6]. This motivates approaches that recover spatial and tactical information directly from broadcast footage.

2.2.2 Academic Benchmarks and Integrated Pipelines

The SoccerNet ecosystem provides the largest open benchmark suite for broadcast football. SoccerNet-v2 introduced annotations for action spotting, camera-shot segmentation, and replay detection [7]; SoccerNet-Tracking added bounding-box annotations and standard evaluation protocols for multi-object tracking [5]; and SoccerNet Game State Reconstruction frames the problem as recovering the full game state on

a top-down minimap, jointly evaluating detection, tracking, identification, and pitch registration [38].

Integrated academic systems address overlapping subsets of this pipeline. Theiner et al. [41] extract positional data by combining detection, tracking, and pitch registration, but evaluate only positional accuracy rather than downstream tactical reasoning. Magera et al. [28] (BroadTrack) target broadcast camera calibration specifically, leaving downstream stages out of scope. Maglo et al. [29] pair field registration with self-supervised re-identification to localise players from a single moving view. Earlier work by Beetz et al. [3] demonstrated the feasibility of automated game analysis but predates modern deep-learning detectors and trackers.

A consistent pattern is that these systems stop at intermediate representations — bounding boxes, tracks, registered pitches, or minimap reconstructions. A reconstruction may be visually plausible while still containing localisation, identity, or ball-detection errors that propagate into derived quantities such as possession, pass detection, distance estimates, or offside decisions. Evaluation at the reconstruction level alone therefore does not establish tactical usefulness.

2.2.3 Positioning of This Project

The literature thus leaves a clear gap: closed commercial systems produce tactical outputs but are not reproducible, while open academic systems are reproducible but stop short of tactical interpretation. This project addresses that gap by extending an open broadcast pipeline beyond reconstruction, treating detection, tracking, and pitch registration as inputs to downstream pitch-space statistics rather than as final outputs.

2.3 Machine Learning and Computer Vision Foundations

This section introduces the machine learning and computer vision concepts on which the project depends, so that the project-specific methods in later sections are accessible to a reader without prior familiarity with these techniques.

2.3.1 Supervised Learning for Visual Recognition

In supervised learning, a model learns from labelled examples. A training set

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$$

pairs each input x_i — for instance an image or image crop — with a label y_i, which may be an object class, a bounding box, a set of keypoints, or a combination of these. The model is a function f_θ with learnable parameters θ, and training minimises the empirical risk

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_\theta(x_i), y_i),$$

where $\mathcal{L}$ measures the discrepancy between prediction and ground truth. Two losses recur in this project. Cross-entropy is used for classification targets,

$$\mathcal{L}_{\mathrm{CE}}(\hat{y}, y) = -\sum_{c=1}^{C} y_c \log \hat{y}_c,$$

where $\hat{y}$ is a predicted distribution over C classes and y is a one-hot ground truth. Squared error is used for continuous targets such as bounding-box coordinates,

$$\mathcal{L}_{\mathrm{MSE}}(\hat{y}, y) = \|\hat{y} - y\|_2^2.$$

Optimisation is performed by stochastic gradient descent or one of its variants, with parameters updated as

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L},$$

where η is the learning rate [12]. The supervised components of this project — the object detector and the pitch keypoint detector — all instantiate this framework, learning visual patterns from annotated examples rather than from hand-written rules.

2.3.2 Neural Networks

A neural network is a parametric model composed of layers of simple computational units. Each unit combines its inputs through learned weights and biases and applies a non-linear activation. For layer l,

$$\mathbf{h}^{(l)} = \sigma\left(W^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right),$$

where $W^{(l)}$ and $\mathbf{b}^{(l)}$ are the layer's learnable parameters and σ is an element-wise non-linearity. The non-linearity is essential: a composition of linear maps is itself linear, so without σ the entire network would collapse to a single linear transformation. The most common choice is the rectified linear unit,

$$\mathrm{ReLU}(z) = \max(0, z),$$

which is computationally cheap and tends to produce sparse activations that aid optimisation [12]. A fully-connected network of this form, however, treats an image as a flat vector — which scales poorly with image size and discards spatial structure. Convolutional architectures, introduced next, address both issues.

2.3.3 Convolutional Neural Networks

Convolutional neural networks (CNNs) are designed for grid-structured data such as images. Rather than connecting every input to every output, a CNN applies small learned filters across local regions of the image, so the same visual pattern — an edge, a texture, a goalpost — can be detected regardless of its position in the frame. A two-dimensional convolutional layer computes

$$y_{i,j,k} = \sigma\left(\sum_{u,v,c} W_{u,v,c,k}\, x_{i+u,\, j+v,\, c} + b_k\right),$$

where x is the input feature map, W is a learned filter, b_k is a bias, and $y_{i,j,k}$ is the output at spatial location (i, j) on output channel k; the indices u, v range over the filter's spatial extent and c over input channels. Weight sharing across spatial positions makes CNNs both more parameter-efficient than fully-connected networks and translation-equivariant by construction. Figure 2.1 illustrates this structure.

Pooling layers reduce spatial resolution and enlarge the effective receptive field of subsequent filters. Max pooling, the most common variant, takes

$$y_{i,j} = \max_{(u,v)\in\mathcal{R}_{i,j}} x_{u,v}$$

over a local region $\mathcal{R}_{i,j}$. Stacking convolution, activation, and pooling yields a hierarchical representation: early layers respond to local patterns such as edges and colour gradients, while deeper layers compose these into semantic features such as body parts, kit textures, or pitch markings. This compositional behaviour was already

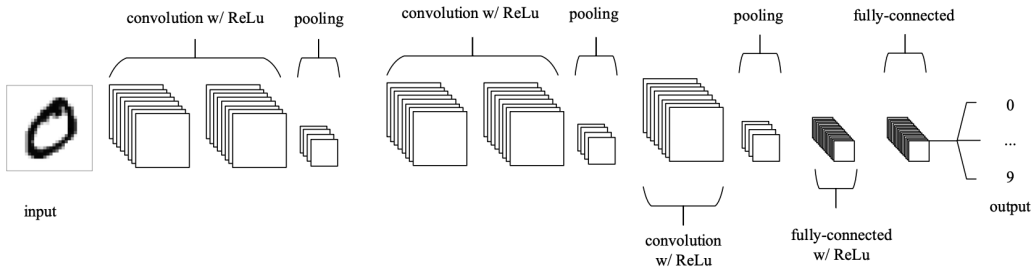


Figure 2.1: A common CNN architecture: stacked convolution, activation, and pooling stages produce feature maps of decreasing spatial resolution, which are passed to fully connected layers for prediction. Reproduced from O'Shea and Nash [32].

present in early systems such as LeNet [19], and deep CNNs became dominant in computer vision after their breakthrough on large-scale image classification [17]. In this project, CNN backbones provide the visual features for both object detection and pitch keypoint localisation; the detection-specific architecture of YOLO is discussed in Section 2.4.

2.3.4 Transfer Learning and Fine-Tuning

Training deep visual models from scratch typically requires very large datasets and substantial computation. Transfer learning sidesteps this by initialising the target-task model with parameters θ_0 learned from a large general-purpose dataset such as ImageNet or COCO. Empirically, the lower layers of such pre-trained networks encode general visual features — edges, colour transitions, textures — that transfer well across domains, while upper layers are more task-specific [43]. Fine-tuning continues training from θ_0 on the target task, often with a smaller learning rate so that useful low-level features are preserved while higher-level representations adapt. This regime is well-suited to broadcast football analysis: labelled football data are scarce, but pre-training already supplies robust features that generalise across the camera-angle, scale, lighting, and motion-blur variations characteristic of broadcast video.

2.4 Object Detection in Broadcast Football

Object detection is the task of locating and classifying objects within an image. In broadcast football the relevant objects are players, goalkeepers, referees, and the ball, and errors at this stage propagate through the rest of the pipeline: missed players degrade tracking, inaccurate boxes corrupt pitch projection, and missed ball detections

undermine possession and pass reasoning.

Formally, a detector maps an input image I to a set of predictions

$$D = \{(b_i, c_i, s_i)\}_{i=1}^{N}, \tag{2.2}$$

where b_i is a bounding box, c_i is the predicted class label, and $s_i \in [0,1]$ is a confidence score reflecting the model's certainty that an object of class c_i is present at b_i.

2.4.1 Bounding Boxes, IoU, and Non-Maximum Suppression

A bounding box is a rectangular region enclosing an object, parameterised either by corner coordinates or by centre, width, and height. The standard measure of localisation quality between a predicted box B_p and a ground-truth box B_g is the Intersection over Union (IoU), illustrated in Figure 2.2,

$$\mathrm{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}, \tag{2.3}$$

which takes values in $[0,1]$, with 1 indicating perfect overlap. A prediction is counted as a true positive only if its IoU with an unmatched ground-truth box exceeds a chosen threshold and the predicted class is correct [21].

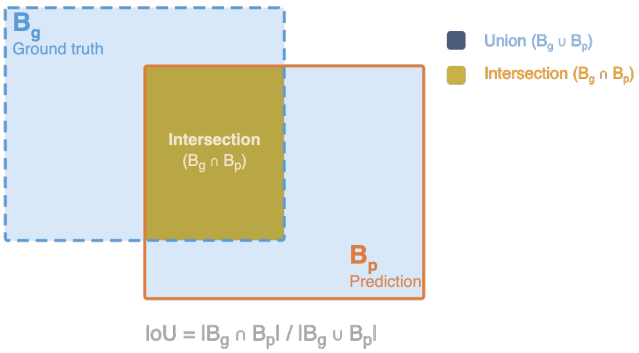


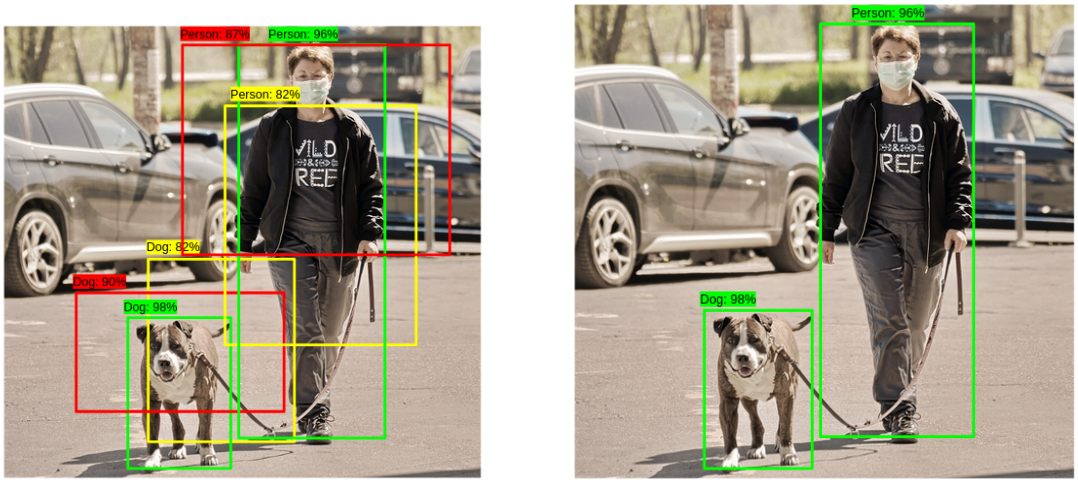
Figure 2.2: Intersection over Union (IoU) between a ground-truth bounding box B_g (blue, dashed) and a predicted bounding box B_p (orange). IoU is the ratio of the intersection area (gold) to the total union area of both boxes, computed as $\mathrm{IoU} = |B_g \cap B_p| / |B_g \cup B_p|$.

Detectors typically emit several overlapping predictions for the same object. Non-Maximum Suppression (NMS) reduces these to a single detection per object via a

greedy procedure: given a candidate set $\mathcal{B}$ of scored predictions and an overlap threshold τ_{NMS}, at each step

$$b^* = \arg\max_{b \in \mathcal{B}} s(b),$$

the highest-scoring remaining detection b^* is retained and any $b \in \mathcal{B}$ with $\text{IoU}(b, b^*) \geq \tau_{\text{NMS}}$ is discarded; the procedure repeats until $\mathcal{B}$ is empty [34]. Figure 2.3 illustrates this process. NMS matters in football footage because players cluster during tackles, set-pieces, and crowded midfield phases — without it, a single player can be counted multiple times and generate duplicate tracks downstream.



(a) Before NMS

(b) After NMS

Figure 2.3: Illustration of Non-Maximum Suppression. Before NMS, multiple overlapping candidate boxes may be produced for the same object. After NMS, lower-confidence overlapping boxes are suppressed and the most confident detection is retained.

2.4.2 Precision, Recall, and Mean Average Precision

Once detections have been matched to ground truth, performance is summarised by precision and recall. Precision measures how many predicted detections are correct, while recall measures how many ground-truth objects are successfully detected:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{2.4}$$

where TP, FP, and FN denote true positives, false positives, and false negatives. Varying the confidence threshold traces out a precision–recall curve: raising the threshold

improves precision at the cost of recall, and vice versa.

Average Precision (AP) summarises this curve as the area beneath it,

$$\mathrm{AP} = \int_0^1 p(r)\, dr,$$

typically approximated by interpolation over a finite set of recall points. Mean Average Precision (mAP) averages AP across classes,

$$\mathrm{mAP} = \frac{1}{C} \sum_{c=1}^{C} \mathrm{AP}_c.$$

Two variants matter for this project. mAP_{50} fixes the IoU threshold at 0.50, while $\mathrm{mAP}_{50\text{-}95}$ averages mAP over IoU thresholds from 0.50 to 0.95 in steps of 0.05 [21] and is therefore a stricter measure of localisation. The distinction is operationally important: a loosely localised player box may suffice for visualisation but not for tracking association or pitch projection.

2.4.3 The YOLO Architecture

Object detectors fall broadly into two families. Two-stage detectors such as Faster R-CNN [36] first propose candidate regions and then classify and refine them, giving strong localisation at the cost of speed. Single-stage detectors such as SSD [24] and YOLO [34] predict boxes and class scores in a single forward pass, making them better suited to video-rate applications such as broadcast football analysis.

YOLO (*You Only Look Once*) treats detection as a single regression problem. The image is divided into a grid, and each cell is responsible for predicting any object whose centre falls inside it: a bounding box, an *objectness* score reflecting whether an object is present at all, and a distribution over class labels [34]. This one-shot design makes YOLO fast, but its accuracy depends critically on how the network is structured to handle objects of very different sizes — in broadcast football, a near-camera player and a far-touchline ball can differ in pixel area by two orders of magnitude.

Architecture. A modern YOLO-style detector consists of three stages, illustrated in Figure 2.4. A convolutional *backbone* processes the image and produces feature maps at several spatial resolutions: shallower maps preserve fine spatial detail useful for small objects, while deeper maps carry richer semantic information useful for classification. A *neck*, typically based on feature pyramid networks [20], fuses these

maps so that every prediction is informed by both fine detail and broader context. The *detection head* reads from the fused maps and outputs, at each spatial location, a bounding box, an objectness score, and class probabilities. A confidence threshold and Non-Maximum Suppression then reduce these per-location predictions to the final detection set.

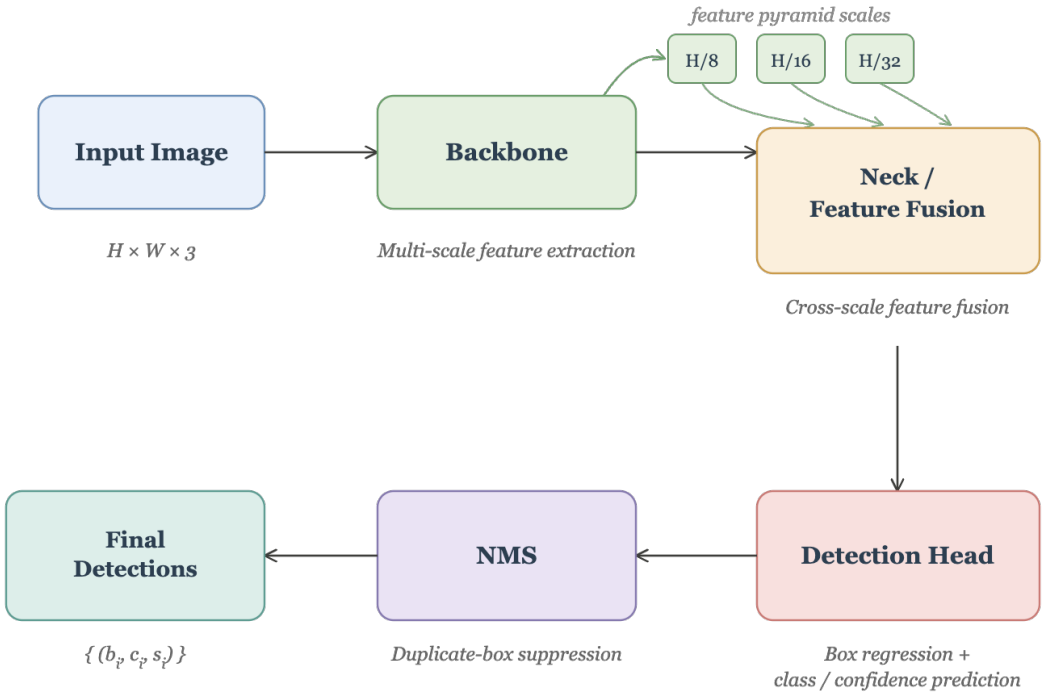


Figure 2.4: Generalised YOLO-style single-stage detection pipeline. The backbone extracts multi-scale feature maps, the neck fuses them, and the detection head predicts boxes, objectness, and class scores. NMS removes duplicate predictions to produce the final detection set $\{(b_i, c_i, s_i)\}$.

How boxes are predicted. Rather than predict absolute pixel coordinates, the detection head predicts *adjustments* relative to a known reference, which is more numerically stable to train. In the original anchor-based design, each spatial location in the feature map is associated with a set of *anchor boxes* — prior guesses of likely object shapes, with width p_w and height p_h. For an anchor at grid-cell offset (c_x, c_y), the network predicts four numbers (t_x, t_y, t_w, t_h) that are decoded into a bounding box

(b_x, b_y, b_w, b_h) by

$$b_x = \sigma(t_x) + c_x, \qquad b_y = \sigma(t_y) + c_y,$$

$$b_w = p_w\, e^{t_w}, \qquad b_h = p_h\, e^{t_h},$$

where σ denotes the sigmoid function [35]. The sigmoid keeps the predicted centre inside its grid cell, and the exponential keeps the predicted width and height positive while expressing them as multiplicative deviations from the anchor's prior shape. Anchor-*free* variants used in YOLOv8 [15] drop the priors (p_w, p_h) altogether and instead regress the four sides of the box directly as distances from each feature-map location — a simpler design that removes a hand-tuned hyperparameter.

Training objective. YOLO is trained to minimise a weighted sum of three losses, one for each output of the head:

$$\mathcal{L} = \lambda_{\text{box}}\, \mathcal{L}_{\text{box}} + \lambda_{\text{obj}}\, \mathcal{L}_{\text{obj}} + \lambda_{\text{cls}}\, \mathcal{L}_{\text{cls}}.$$

The first term $\mathcal{L}_{\text{box}}$ penalises localisation error, typically using an IoU-based loss such as CIoU. The second $\mathcal{L}_{\text{obj}}$ is a binary loss over whether each predicted location actually contains an object. The third $\mathcal{L}_{\text{cls}}$ is a classification loss over the object class. Splitting objectness from classification lets the model learn the question *is something here?* independently of *what is it?*, which helps when classes overlap visually — as players and goalkeepers do in broadcast football.

Relevance to broadcast football. Multi-scale feature extraction is what makes YOLO viable on broadcast footage. Player sizes span a wide range within any single frame, and the ball is often only a handful of pixels across. By combining fine- and coarse-resolution feature maps in a single network, the detector can localise objects across this entire scale range.

2.5 Multi-Object Tracking

Object detection produces independent bounding boxes in each frame, but does not say whether a player detected in one frame is the same player detected in the next. Multi-object tracking (MOT) addresses this by linking detections over time and assigning a persistent identity to each object [8]. Tracking is essential to this project because every downstream output — heatmaps, distance and speed estimates, team assignment,

possession reasoning, and offside analysis — requires following the same player across frames.

Formally, let

$$D_t = \{d_t^1, \ldots, d_t^{n_t}\} \tag{2.5}$$

denote the detections at frame t, each carrying a bounding box, class, and confidence, and let

$$\mathcal{T}_{t-1} = \{\tau_{t-1}^1, \ldots, \tau_{t-1}^{m_{t-1}}\} \tag{2.6}$$

denote the active tracks from the previous frame. Tracking must assign each $d \in D_t$ to a track in $\mathcal{T}_{t-1}$, create new tracks for unmatched detections, and gracefully handle tracks for which no matching detection arrives.

2.5.1 Tracking-by-Detection

Most modern MOT systems follow the *tracking-by-detection* paradigm [4]: an object detector is applied independently to each frame, and a separate tracker associates the resulting detections over time. This decouples visual recognition from temporal association — the detector answers *what is in this frame?* while the tracker answers *which detections belong to the same object?* The framework is modular (detector improvements can be dropped in without redesigning the tracker) and online (associations are produced frame-by-frame, without waiting for the full sequence). Its principal weakness is detector dependence: missed detections fragment tracks, false positives spawn spurious tracks, and inaccurate boxes produce incorrect associations — failures that cluster in crowded scenes such as set-pieces and midfield contests.

2.5.2 Motion Prediction and Data Association

A standard motion model is the Kalman filter, which recursively estimates the state of a linear-Gaussian system from noisy observations [16]. For a tracked bounding box, a common state under a constant-velocity assumption is

$$\mathbf{x}_t = [x_t, y_t, w_t, h_t, \dot{x}_t, \dot{y}_t, \dot{w}_t, \dot{h}_t]^\top,$$

combining the box centre, its width and height, and their rates of change. The filter alternates between two steps. The *predict* step propagates the previous state ($\mathbf{x}_{t-1}$) and

its covariance (P_{t-1}) forward,

$$\hat{\mathbf{x}}_t^- = F\mathbf{x}_{t-1}, \qquad P_t^- = FP_{t-1}F^\top + Q, \tag{2.7}$$

where F is the state-transition matrix encoding the motion model and Q is the process-noise covariance. The predicted state $\hat{\mathbf{x}}_t^-$ gives the expected location of the track before seeing the detector output for frame t. The *update* step then corrects this prediction using a matched detection $\mathbf{z}_t$:

$$K_t = P_t^- H^\top \left(HP_t^- H^\top + R\right)^{-1}, \tag{2.8}$$

$$\mathbf{x}_t = \hat{\mathbf{x}}_t^- + K_t\left(\mathbf{z}_t - H\hat{\mathbf{x}}_t^-\right), \qquad P_t = (I - K_t H)\,P_t^-, \tag{2.9}$$

where H is the observation matrix, R is the measurement-noise covariance, and the Kalman gain K_t weighs the new evidence against the prior prediction, trusting the detection more when measurement noise is small relative to predicted state uncertainty. The term $\mathbf{z}_t - H\hat{\mathbf{x}}_t^-$ is the difference between the observed detection and the predicted measurement.

Given predicted track locations, association reduces to a bipartite matching problem in which each track receives at most one detection and vice versa. We construct a cost matrix $C \in \mathbb{R}^{m\times n}$ with

$$C_{ij} = 1 - \mathrm{IoU}(\hat{B}_i, B_j), \tag{2.10}$$

where $\hat{B}_i$ is the predicted box for track i and B_j is the box of detection j; lower cost means better match. The Hungarian algorithm solves this assignment problem to optimality in $O(n^3)$ time [18]. Unmatched detections seed new tracks; unmatched tracks are buffered and eventually deleted if not recovered. This Kalman–Hungarian skeleton underlies SORT [4], ByteTrack [45], and BoT-SORT [2].

2.5.3 ByteTrack

ByteTrack [45] observes that low-confidence detections are not all false: confidence frequently drops during occlusion, motion blur, or truncation while the object remains present. Discarding them therefore loses useful evidence. ByteTrack instead partitions detections by confidence,

$$D_t^{\text{high}} = \{d \in D_t : s_d \geq \tau_{\text{high}}\}, \quad D_t^{\text{low}} = \{d \in D_t : s_d < \tau_{\text{high}}\}, \tag{2.11}$$

where s_d is the confidence score of detection d, τ_{high} is the high-confidence threshold, and runs association in two passes (Figure 2.5). Tracks are first matched against D_t^{high} using the Hungarian algorithm with IoU cost; any track that remains unmatched is then given a second chance against D_t^{low}. High-confidence detections that fail to match seed new tracks; tracks unmatched after both passes are held in a buffer for a short period before deletion. This design preserves track continuity through short occlusions and detector dropouts while still anchoring associations to high-confidence evidence.

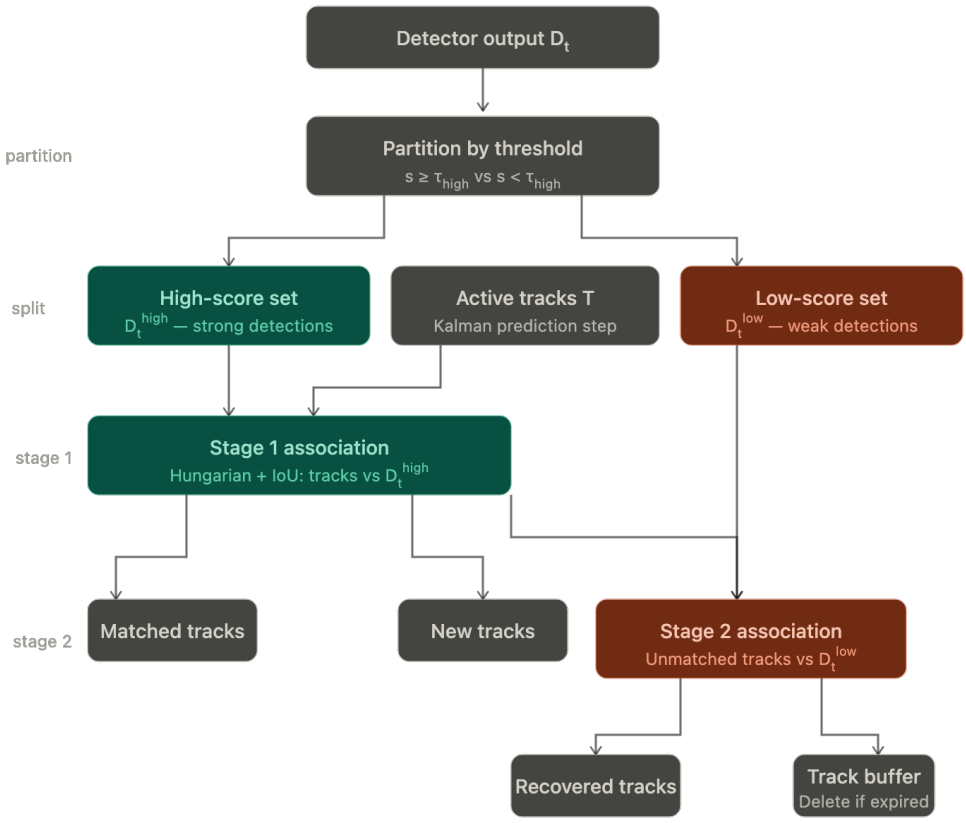


Figure 2.5: ByteTrack two-stage association process. Detector output is partitioned by confidence threshold τ_{high} into a high-score set D_t^{high} and a low-score set D_t^{low}. Stage 1 matches active tracks against D_t^{high} using the Hungarian algorithm with IoU cost; unmatched tracks proceed to Stage 2, where they are compared against D_t^{low}. Tracks that remain unmatched after both stages are held in a buffer and deleted if unrecovered within a fixed number of frames.

2.5.4 BoT-SORT

BoT-SORT [2] extends the tracking-by-detection skeleton with two refinements aimed at challenging video. First, *camera-motion compensation*: when the camera pans, tilts, or zooms, all bounding-box positions shift together, and naïve IoU between predicted and observed boxes becomes unreliable. BoT-SORT estimates a global affine transform between consecutive frames — typically by matching sparse keypoints in the static background — and pre-warps each predicted track location into the current frame's coordinates before association. Second, *appearance-based re-identification*: visual features extracted from detection crops disambiguate spatially similar candidates, particularly during occlusions. Both refinements add per-frame computational cost relative to ByteTrack.

2.5.5 Tracking Evaluation: HOTA

Tracking quality must capture two properties simultaneously: how accurately objects are localised, and how consistently their identities persist over time. Multi-Object Tracking Accuracy (MOTA) and Identification F1 (IDF1) are established baselines [8], but both conflate detection and association in ways that obscure the source of errors.

Higher Order Tracking Accuracy (HOTA) addresses this by decomposing performance into separate detection and association scores [26]:

$$\mathrm{HOTA}_\alpha = \sqrt{\mathrm{DetA}_\alpha \cdot \mathrm{AssA}_\alpha}, \tag{2.12}$$

where DetA_α measures box overlap with ground truth at localisation threshold α, and AssA_α measures identity consistency — the average proportion of ground-truth and predicted trajectories that share an identity over their joint lifetime. The geometric mean ensures that neither component can dominate: a tracker that localises well but fragments identities, or associates well but misses many objects, will be penalised in both factors. The final HOTA score averages Equation 2.12 over $\alpha \in [0.05, 0.95]$, giving a single threshold-independent summary.

2.6 Team Identification from Visual Features

Object detection and tracking determine where players are and how they move, but neither directly recovers which team each player belongs to. Team identity is not

produced by a generic detector, and manually labelling every player per match would defeat the purpose of an automated pipeline. Because players on the same team share a visually similar kit, the task is naturally cast as an unsupervised grouping problem on player appearance [14].

2.6.1 Team Identification as Clustering

Each detected player crop I_i is mapped to a feature vector

$$\mathbf{z}_i = f(I_i), \qquad \mathbf{z}_i \in \mathbb{R}^d, \tag{2.13}$$

by a feature extractor f chosen so that crops of players in the same kit produce similar vectors and crops of opposing players produce dissimilar ones. The team-identification problem is then to partition $\{\mathbf{z}_i\}_{i=1}^{N}$ into groups corresponding to the teams on the pitch — a standard unsupervised clustering problem [27]. For the two outfield teams the natural choice is two clusters, since their kits are required by competition rules to be visually distinguishable.

2.6.2 Visual Representations

The classical approach extracts colour histograms in spaces such as RGB, HSV, or Lab and clusters players by histogram similarity [14]. Such features are fast, interpretable, and align with the cue humans use, but they are sensitive to lighting changes, shadows, and background pixels included in the crop, and they discard spatial structure entirely.

A more robust alternative is to use deep visual embeddings. Modern pretrained encoders such as CLIP, SigLIP, and DINOv2 learn transferable visual representations from internet-scale pre-training [33, 44, 31] and produce embeddings that capture colour, texture, shape, and pattern jointly in a single vector. Because their training data covers extensive visual variation, the resulting features generalise to new domains — including broadcast football crops — without any domain-specific fine-tuning, which makes them well-suited to a setting in which match-specific labels are unavailable.

For clustering, embeddings ($\mathbf{z}_i$) are typically L2-normalised so that

$$\tilde{\mathbf{z}}_i = \mathbf{z}_i / \|\mathbf{z}_i\|_2,$$

and similarity is measured by cosine similarity,

$$\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_j) = \frac{\mathbf{z}_i^\top \mathbf{z}_j}{\|\mathbf{z}_i\|_2 \, \|\mathbf{z}_j\|_2}.$$

On the unit sphere, cosine similarity and squared Euclidean distance differ only by a constant — $\|\tilde{\mathbf{z}}_i - \tilde{\mathbf{z}}_j\|_2^2 = 2 - 2\tilde{\mathbf{z}}_i^\top \tilde{\mathbf{z}}_j$ — so Euclidean-distance clustering on normalised embeddings is equivalent to clustering by cosine similarity. This makes pretrained embeddings directly usable with the distance-based clustering algorithms below.

2.6.3 Dimensionality Reduction

Deep embeddings are typically high-dimensional ($d \in [512, 1024]$ for the encoders cited above), and distance-based clustering becomes less stable in such spaces because pairwise distances concentrate — many points appear nearly equidistant, a phenomenon known as the curse of dimensionality [1]. Dimensionality reduction maps embeddings into a lower-dimensional space $\mathbb{R}^{d'}$ with $d' \ll d$ before clustering:

$$\phi : \mathbb{R}^d \to \mathbb{R}^{d'}, \qquad \mathbf{y}_i = \phi(\mathbf{z}_i). \tag{2.14}$$

Principal Component Analysis (PCA) is the classical linear choice. Given centred embeddings stacked as $Z \in \mathbb{R}^{N \times d}$, PCA computes the eigendecomposition of the sample covariance matrix $\Sigma = \frac{1}{N} Z^\top Z$ and projects onto the top-d' eigenvectors of Σ, yielding the linear subspace that maximises preserved variance. PCA is fast and deterministic but its linearity means it cannot unfold curved manifold structure that embedding models often produce.

UMAP (Uniform Manifold Approximation and Projection) is a non-linear alternative that constructs a weighted neighbourhood graph in the high-dimensional space and optimises a low-dimensional layout to preserve its local topology [30]. The objective minimises the cross-entropy between the high- and low-dimensional neighbour distributions, which preserves the local similarity structure that matters for team identification: players in the same kit should remain close, even when their global arrangement on a manifold is non-linear.

2.6.4 k-Means Clustering

Given reduced features $\{\mathbf{y}_i\}_{i=1}^{N}$, k-means partitions them into k clusters by minimising the within-cluster sum of squared distances [27]:

$$\min_{\{\mu_j\}_{j=1}^{k}} \sum_{i=1}^{N} \min_{1\le j\le k} \|\mathbf{y}_i - \mu_j\|_2^2, \tag{2.15}$$

where μ_j is the centroid of cluster j. This objective is non-convex, so direct minimisation is intractable. In practice it is solved by Lloyd's algorithm, which alternates two steps until convergence. Given current centroids $\{\mu_j\}$, each point is assigned to its nearest centroid,

$$a_i = \arg\min_{1\le j\le k} \|\mathbf{y}_i - \mu_j\|_2^2,$$

and each centroid is then updated as the mean of the points assigned to it,

$$\mu_j = \frac{1}{|\{i : a_i = j\}|} \sum_{i:a_i=j} \mathbf{y}_i.$$

Each step strictly decreases or preserves the objective, guaranteeing convergence to a local minimum. The local optimum reached depends on initialisation; k-means++ provides a probabilistic seeding scheme that selects initial centroids far apart in feature space, giving provably better expected performance than uniform random initialisation. For team identification on outfield players $k = 2$, and the algorithm recovers the two groups up to an arbitrary label permutation — separating *which player is on which team*, but not *which cluster corresponds to which named team*.

2.7 Pitch Registration and Planar Homography

Detections and tracks are produced in image coordinates, but tactical analysis — bird's-eye visualisation, heatmaps, distance and speed estimation, possession analysis, offside reasoning — requires positions in a common pitch coordinate system. Pitch registration estimates a geometric mapping between the broadcast image and a canonical top-down pitch model, allowing image-space observations to be projected into pitch space [41, 40].

2.7.1 Planar Homography

The football pitch is approximately planar, and projective geometry tells us that any two views of a plane are related by a 3×3 projective transformation called a *planar homography* [13]. Working in homogeneous coordinates, an image point $\tilde{\mathbf{x}} = (u, v, 1)^\top$ and its corresponding pitch-space point $\tilde{\mathbf{X}} = (x, y, 1)^\top$ are related by

$$\tilde{\mathbf{X}} \sim H\tilde{\mathbf{x}}, \tag{2.16}$$

where $\sim$ denotes equality up to a non-zero scale factor. Because H is defined only up to scale, it has eight degrees of freedom rather than nine. Writing

$$H = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{pmatrix},$$

the explicit pitch coordinates of an image point are

$$x = \frac{h_{11}u + h_{12}v + h_{13}}{h_{31}u + h_{32}v + h_{33}}, \qquad y = \frac{h_{21}u + h_{22}v + h_{23}}{h_{31}u + h_{32}v + h_{33}}, \tag{2.17}$$

which makes the projective nature explicit: the denominator is what produces perspective foreshortening, and reduces to an affine transformation only when $h_{31} = h_{32} = 0$.

The mapping is valid only for points lying on (or very near) the pitch plane. Player feet are therefore the correct reference point for projection, and the bottom centre of each player's bounding box serves as a practical approximation to the foot location.

2.7.2 Estimating H from Point Correspondences

Estimating H requires correspondences $\{(\mathbf{x}_i, \mathbf{X}_i)\}_{i=1}^{N}$ between image and pitch points. Each correspondence yields *two* independent constraints on H: from Equation 2.16, the cross product $\tilde{\mathbf{X}}_i \times H\tilde{\mathbf{x}}_i = \mathbf{0}$ gives three scalar equations, of which only two are linearly independent. Since H has eight degrees of freedom, at least four non-collinear correspondences are required to determine it; with more than four, the system becomes overdetermined.

The standard solution is the Direct Linear Transform (DLT) algorithm [13]. Stacking the two constraints per correspondence produces a homogeneous linear system

$$A\mathbf{h} = \mathbf{0}, \qquad A \in \mathbb{R}^{2N \times 9}, \tag{2.18}$$

where $\mathbf{h} \in \mathbb{R}^9$ is the vectorised form of H. The least-squares solution under the constraint $\|\mathbf{h}\|_2 = 1$ is the right singular vector of A corresponding to its smallest singular value, obtained from the singular value decomposition of A. In practice, image and pitch coordinates are first normalised — typically translated to have zero mean and scaled to have unit average distance from the origin — to improve numerical conditioning [13] before solving.

Real correspondences contain both noise and outliers — incorrectly localised keypoints whose error is large enough to corrupt the least-squares estimate. Robust estimation is therefore performed using RANSAC, which repeatedly samples minimal sets of four correspondences, fits a candidate homography from each, and counts how many of the remaining correspondences are consistent with it (within a chosen reprojection threshold). The homography with the largest consensus set is retained, and the final estimate is refined by re-running DLT on its inliers only. RANSAC tolerates outlier rates well above 50%, which makes it the standard choice when correspondences come from an automatic detector rather than from manual annotation.

2.7.3 Pitch Keypoint Detection

The correspondences required by DLT are obtained by detecting semantic landmarks on the playing surface — line intersections, penalty-area corners, centre-circle tangencies, and similar distinctive points formed by pitch markings — whose pitch-space coordinates are fixed by the laws of the game and therefore known a priori. Detecting these landmarks in the broadcast image yields exactly the $(\mathbf{x}_i, \mathbf{X}_i)$ pairs that Equation 2.18 requires.

This is structurally a keypoint detection problem and is solved by the same family of CNN architectures used for human pose estimation, with the target points replaced by fixed pitch landmarks. A convolutional backbone extracts multi-scale feature maps and a prediction head outputs per-landmark confidences and locations [15]. Keypoint quantity, accuracy, and spatial spread all matter: more correspondences overdetermine the system and average out localisation noise; correspondences distributed across the visible pitch constrain the transformation more tightly than correspondences clustered in one region.

2.7.4 Reprojection Error

Registration quality is measured by reprojection error. For correspondence i,

$$e_i = \|\mathbf{X}_i - \pi(H\tilde{\mathbf{x}}_i)\|_2 , \tag{2.19}$$

where $\pi(\cdot)$ converts homogeneous coordinates back to Cartesian by dividing by the third component. The mean reprojection error over N correspondences,

$$\bar{e} = \frac{1}{N}\sum_{i=1}^{N} e_i, \tag{2.20}$$

summarises how closely the estimated H maps the detected landmarks back to their known pitch locations.

Low reprojection error is necessary but not sufficient for accurate downstream projection. The metric is computed only on the keypoints used to fit H, so a homography that fits its inputs perfectly can still extrapolate poorly to regions of the pitch where no landmarks were detected — particularly when keypoints cluster in one half of the image, leaving the other half geometrically underconstrained.

2.8 Spatial and Tactical Analysis in Pitch Space

Once detections and tracks have been mapped to a canonical pitch model, analysis can proceed in a coordinate system that is independent of the broadcast camera. This matters because pixel distances do not correspond uniformly to real-world distances under projective viewing: equal displacements in image space can encode very different real displacements depending on where they occur in the frame. Pitch-space coordinates remove this distortion, making it possible to compare positions, compute movement statistics, and reason about tactical events in a common geometric frame [41, 11].

2.8.1 Pitch-Space Trajectories

Each tracked player must be reduced to a single image point before the homography is applied. Any such point must lie on the ground plane, since the homography is valid only there; an image point above the plane projects incorrectly because its line of sight does not intersect the pitch where the player stands.

Let $\mathbf{q}_t^{(i)} \in \mathbb{R}^2$ be the chosen ground-plane image point for player i at frame t. Given

an image-to-pitch homography H, the projected pitch-space position is

$$\mathbf{p}_t^{(i)} = \pi\Big(H\tilde{\mathbf{q}}_t^{(i)}\Big), \tag{2.21}$$

where $\tilde{\mathbf{q}}_t^{(i)}$ is the homogeneous form of the image point and $\pi(\cdot)$ converts back to Cartesian coordinates. The sequence $\{\mathbf{p}_t^{(i)}\}_{t=1}^{T}$ is the pitch-space trajectory of player i, and is the geometric primitive on which all subsequent spatial analysis is built.

2.8.2 Heatmap Construction

A heatmap summarises the spatial distribution of a player's or team's positions over time. The pitch is divided into a regular grid, and each projected position contributes to the cell containing it:

$$H_i(u,v) = \sum_{t=1}^{T} \mathbf{1}\Big[(u,v) = \mathrm{bin}\Big(\mathbf{p}_t^{(i)}\Big)\Big], \tag{2.22}$$

where $\mathbf{1}[\cdot]$ is the indicator function and $\mathrm{bin}(\cdot)$ maps a continuous pitch coordinate to a discrete grid cell. Equation 2.22 is a two-dimensional histogram of the trajectory, and a team-level heatmap is obtained by accumulating positions from all players assigned to that team.

Histogram estimates are noisy, particularly when sample counts per cell are small. Smoothing with an isotropic Gaussian kernel,

$$\tilde{H}_i = H_i * G_\sigma, \tag{2.23}$$

yields a kernel density estimate of the underlying spatial distribution. The bandwidth σ controls the bias–variance trade-off: a larger σ suppresses noise but blurs spatial detail; a smaller σ preserves detail at the cost of higher variance per cell.

2.8.3 Speed and Distance Estimation

Movement statistics derive from temporal differences of the trajectory. With a video frame rate f_s and inter-frame interval $\Delta t = 1/f_s$, for player i, a finite-difference velocity estimate at frame t is

$$\mathbf{v}_t^{(i)} = \frac{\mathbf{p}_t^{(i)} - \mathbf{p}_{t-1}^{(i)}}{\Delta t}, \tag{2.24}$$

where $\mathbf{p}_t^{(i)}$ and $\mathbf{p}_{t-1}^{(i)}$ are consecutive pitch-space positions. The corresponding instantaneous speed is

$$s_t^{(i)} = \|\mathbf{v}_t^{(i)}\|_2. \tag{2.25}$$

Total distance covered can then be approximated by summing the Euclidean displacement between consecutive positions:

$$d^{(i)} = \sum_{t=2}^{T} \left\| \mathbf{p}_t^{(i)} - \mathbf{p}_{t-1}^{(i)} \right\|_2. \tag{2.26}$$

Finite differencing is intrinsically noise-amplifying. If each pitch-space position carries independent Gaussian localisation noise with variance σ_p^2, the variance of Equation 2.24 is $2\sigma_p^2/\Delta t^2$, so a small per-frame localisation error becomes a large per-frame velocity error. This is a property of the differencing operator itself rather than of any particular pipeline.

2.8.4 Possession via Nearest-Player Assignment

Possession and passes are not detected directly from a frame. They are inferred from the spatial relationship between the ball, the players, and their team identities over time. The simplest model assigns the ball to the spatially closest eligible player. Given a ball position $\mathbf{b}_t \in \mathbb{R}^2$ and a candidate set $\mathcal{A}_t$ of players' pitch positions $\mathbf{p}_t^{(i)}$, the assigned holder is

$$h_t = \arg\min_{i \in \mathcal{A}_t} \|\mathbf{b}_t - \mathbf{p}_t^{(i)}\|_2. \tag{2.27}$$

This is exactly the assignment of $\mathbf{b}_t$ to a cell of the Voronoi diagram of $\{\mathbf{p}_t^{(i)}\}$, in which each player owns the region of pitch closer to them than to any other player. A pass corresponds to a transition of h_t between distinct players, normally on the same team.

A more refined formulation is *pitch control*, in which each player exerts a continuous influence over the pitch as a function of position, velocity, and orientation, and the ball is assigned probabilistically to the player with the greatest influence at its current location [10]. Each player i is associated with an influence function $I^{(i)}(\mathbf{r},t)$ — typically a bivariate Gaussian whose mean is offset forward of the player along their velocity vector and whose covariance grows with speed — and the probability that player i controls location $\mathbf{r}$ at time t is

$$P^{(i)}(\mathbf{r},t) = \frac{I^{(i)}(\mathbf{r},t)}{\sum_j I^{(j)}(\mathbf{r},t)}.$$

Pitch control captures dynamic context that nearest-player assignment ignores — momentum, reach, time-to-arrive — but requires reliable velocities and orientations, and therefore higher-quality input than nearest-player assignment.

2.8.5 Offside Rule Formalisation

Offside is fundamentally a spatial rule and therefore admits a pitch-space formalisation. Under Law 11 of the Laws of the Game, a player is in an offside position if, at the moment the ball is played by a team-mate, they are in the opponents' half and nearer to the opponents' goal line than both the ball and the second-last opponent [39].

Choose a coordinate system in which the attacking direction of the team in possession is the positive x-axis. Let $x_t^{(a)}$, $x_t^{(b)}$, and $x_t^{(o_2)}$ be the longitudinal coordinates at the moment of the pass of an attacking player, the ball, and the second-last opponent respectively. The positional offside test is then

$$x_t^{(a)} > \max\left(x_t^{(b)}, x_t^{(o_2)}\right), \tag{2.28}$$

combined with the requirement that the attacker is in the opponents' half. The maximum expresses the rule's "beyond both" condition: the second-last opponent serves as the defensive line, but the ball overrides this when it is ahead of that line.

Equation 2.28 captures positional offside only. Law 11 additionally requires the player to become actively involved in play before being penalised, which is a behavioural rather than purely spatial condition [39]; this falls outside what pitch-space geometry alone can decide. The formalisation nonetheless illustrates how a tactical rule reduces to a geometric inequality once players, ball, and pitch share a coordinate system.

2.9 Chapter Summary

This chapter has developed the theoretical foundation for the broadcast football analysis pipeline. The unifying view is that the system is a composition of stages — detection, tracking, team identification, pitch registration, and pitch-space tactical reasoning — each formalised mathematically and each consuming the outputs of those before it. This structure makes the pipeline modular and individually testable, but also tightly coupled: errors at any stage propagate downstream, so upstream quality bounds what later stages can achieve. The methods, metrics, and notation introduced here are

applied directly in the implementation described in Chapter 3 and in the evaluation presented in Chapter 4.

Chapter 3

Methodology and Implementation

This chapter describes the design and implementation of the broadcast football analysis pipeline introduced in Chapter 1 and grounded in the theory of Chapter 2. Sections are organised to mirror the data flow through the system: each section corresponds to one processing stage f_k of the composed pipeline $y = f_n(\dots f_1(x))$ of Equation 2.1, with the same notation reused throughout. Empirical evaluation against the success criteria of Section 1.2 is deferred to Chapter 4.

3.1 System Overview

The pipeline ingests a broadcast football video and emits four output classes: an annotated MP4 with bounding boxes, team labels, track identifiers, and offside indicators; a bird's-eye-view MP4 in pitch-space coordinates; per-track heatmap PNGs; and JSON logs of pass and offside events together with per-player match statistics. Internally, processing decomposes into three parallel branches that converge at a shared pitch-space representation, illustrated in Figure 3.1.

Three design decisions distinguish this architecture from a single-detector baseline. First, the ball-class outputs of the generic detector are discarded in favour of a dedicated single-class ball model, motivated by the per-class imbalance and small-object difficulty quantified in Section 3.4. Second, goalkeeper team identity is resolved by a spatial heuristic rather than by the unsupervised clusterer, because goalkeeper kits are required by competition rules to be visually distinct from both outfield kits, and a two-cluster model therefore cannot place them reliably (Section 3.6). Third, pitch keypoint inference is the slowest stage of the pipeline; it is run only every fifteen frames and the resulting homography is cached, exploiting the slow camera motion characteristic of

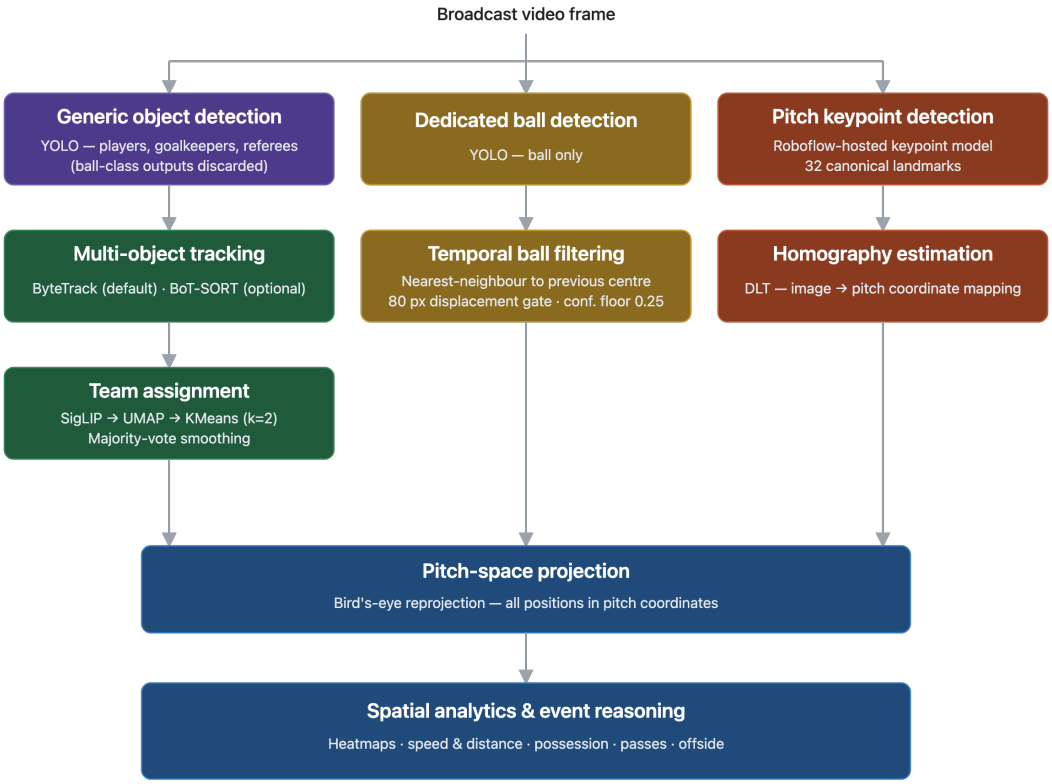


Figure 3.1: Pipeline architecture. Three parallel branches process each frame: a generic object detector with multi-object tracker producing per-frame player and referee detections that are then assigned to teams (left); a dedicated ball detector with a temporal coherence filter (centre); and a pitch keypoint detector whose outputs are used to estimate a per-frame homography by Direct Linear Transform (right). The three branches converge at a shared pitch-space representation, on which all spatial analytics — heatmaps, speed and distance, possession, passes, and offside — are computed.

broadcast football (Section 3.7).

The three branches converge at the pitch-space projection step, where each player's bottom-centre image-space anchor and the ball's position are mapped through the cached homography in line with the ground-plane validity argument of Section 2.8.1. All downstream analytics operate on these pitch-space positions and are detailed in Section 3.8.

A separate *warmup phase* runs once at the start of each video, fitting the unsupervised SigLIP–UMAP–KMeans clustering of Section 2.6.4 on player crops sampled from the first twenty seconds of footage. Warmup is conceptually outside the per-frame loop but operates on the same input video, so the entire pipeline produces team labels without per-match labelling effort, directly addressing the third project objective of Section 1.2.

Table 3.1 records the intermediate representations passed between modules, with each representation labelled by the section in which its producer is defined. The remainder of this chapter follows the order of Figure 3.1.

#	Representation	Contents	Produced by	Consumed by
1	Raw detection	bbox, class, confidence	Generic YOLO detector (§3.4)	Tracker; ball filter
2	Track	bbox, class, conf, track ID	ByteTrack/BoT-SORT (§3.5)	Team classifier; projection
3	Filtered ball detection	bbox, confidence	Dedicated ball detector + temporal filter (§3.4)	Possession tracker
4	Team-labelled track	track ID, raw label, smoothed label	`TeamAssigner` (§3.6)	Possession; offside; renderer
5	Pitch keypoints	image-space coords + per-keypoint confidence	YOLO-pose model (§3.7)	Homography estimator
6	Homography	3×3 image-to-pitch transform	DLT estimator (§3.7)	ViewTransformer
7	Pitch-space position	(x, y) in centimetres on the pitch	ViewTransformer (§3.8)	Heatmaps; speed; possession; offside
8	Event log	passes, offsides, per-track statistics	Tactical reasoning modules (§3.8)	JSON outputs; Ch. 4

Table 3.1: Intermediate representations passed between pipeline components, with the section defining each producer indicated in parentheses.

3.2 Requirements and Design Objectives

The aims and objectives of Chapter 1 commit the system to six functional capabilities and an associated set of qualitative success criteria. Before describing the implementation it is useful to record these commitments in a more contractual form, as functional and non-functional requirements with explicit traceability to the originating objectives and to the implementation sections that deliver them. The full requirements set is given in Table 3.2; subsequent sections of this chapter return to the FR/NFR identifiers when describing how each requirement is met.

3.3 Development Methodology

The project followed a modular, iterative, component-wise development approach. Each processing stage in Figure 3.1 was first implemented and inspected in isolation, with explicit input and output contracts as recorded in Table 3.1, before being integrated into the end-to-end loop. This was a deliberate choice driven by error propagation: tactical outputs depend on the cumulative correctness of detection, tracking, team assignment, and pitch registration, so an under-tested component would fail in ways that surface only at the analytics stage and would be difficult to attribute to its true source. Component isolation kept the failure surface small at each stage of development. End-to-end integration was performed only after each module had been independently verified.

Design decisions for each component were informed first by the literature reviewed in Chapter 2 and then checked empirically on a small set of held-out development clips. The architecture-level choices — the two-detector split, the spatial goalkeeper resolver, the cached homography — were arrived at through this loop: the literature established the available techniques and their known failure modes (Sections 2.4, 2.6, and 2.7), and inspection of intermediate outputs on development footage exposed the specific failure modes that motivated each design departure. The empirical justifications appearing throughout this chapter (most prominently the per-class detection breakdown of Section 3.4) are the diagnostic evidence collected during this development loop, separate from the final quantitative evaluation reported in Chapter 4.

Parameters governing component integration — temporal windows for smoothing, confidence thresholds for filtering, displacement gates for plausibility checks — were calibrated on the same development clips, with the goal of suppressing transient

ID	Requirement	Source	Implemented by
Functional			
FR1	Detect players, goalkeepers, referees, and the ball in broadcast football frames with mAP@50 $\geq$ 0.80.	Obj. 1; SC (i)	Generic four-class detector and dedicated ball detector (§3.4).
FR2	Maintain consistent player and referee identities across frames sufficient for downstream tactical analysis.	Obj. 2; SC (ii)	Multi-object tracker with selectable algorithm (§3.5).
FR3	Assign each tracked player to a team without per-match manual labels.	Obj. 3; SC (iii)	Unsupervised SigLIP–UMAP–KMeans clustering with track-level smoothing and a goalkeeper resolver (§3.6).
FR4	Project image-space detections into pitch-space coordinates with sufficient geometric accuracy to support spatial analytics.	Obj. 4	YOLO-pose pitch keypoint detector and DLT-based homography estimator (§3.7).
FR5	Produce tactical outputs including possession, pass and offside events, per-player speed and distance, and positional heatmaps.	Obj. 5; SC (iv)	Tactical reasoning modules operating in pitch-space (§3.8).
FR6	Operate end-to-end as an integrated pipeline on a complete broadcast clip without manual intervention.	Obj. 6	Pipeline orchestration with two-phase execution and cached intermediates (§3.10).
Non-functional			
NFR1	Components are independently replaceable without modifying unrelated modules.	Obj. 6	Package layout under `src/`; configuration-file tracker selection (§3.10).
NFR2	The system is reproducible from documented model weights, datasets, hyperparameters, and configuration files.	Obj. 6	Reproducibility documentation (§3.10.3).
NFR3	Outputs support both human inspection and machine-readable downstream analysis.	Obj. 5	MP4, PNG visualisations alongside NPY and JSON exports (§3.10).

Table 3.2: Functional (FR) and non-functional (NFR) requirements, with the originating Chapter 1 objective (Obj.) and success criterion (SC) where applicable, and the implementation section that delivers each requirement.

errors without rejecting plausible behaviour. They were not selected by formal hyperparameter search: the project's contribution is in the surrounding architecture and the component-level design decisions, not in fine-grained parameter tuning. Where a parameter is principled (for example, a sprinter-ceiling speed cap or a frame-rate-derived smoothing window) the rationale is stated alongside the value in the section that introduces it. Implementation-level testing performed during this development process is documented in Section 3.9.

3.4 Object and Ball Detection

The pipeline uses two distinct YOLO detectors. A four-class *generic detector* locates players, goalkeepers, referees, and the ball, and a single-class *dedicated ball detector* re-runs ball localisation on every frame. Only the player, goalkeeper, and referee predictions of the generic detector flow into the rest of the pipeline; its ball-class outputs are discarded for the empirical reason developed below. This section covers both models, addressing FR1 and the first project objective of Section 1.2.

3.4.1 Generic Four-Class Detector

The generic detector is a YOLOv8m model [15] fine-tuned on the *Football_object_detection_dataset* (Roboflow), comprising 768 broadcast frames split 618/75/75 across train/validation/test. The dataset contains 18,838 instances across the four classes, distributed unevenly: players account for 15,297 instances — 12.6 times as many as the ball class (1,213) and 26.3 times as many as the goalkeeper class (581). This imbalance has direct consequences for per-class performance.

The medium variant (`yolov8m.pt`) was selected as a compromise between detection accuracy and inference cost. The pipeline is intended to run on commodity hardware without a discrete GPU, so a larger backbone would dominate end-to-end runtime even though it might add a small amount of accuracy. Inputs were resized to 1280×1280 to preserve the small-object detail required for ball localisation, which constrained training-time batch size to 4 on the available NVIDIA A100 GPU. The model was trained for 50 epochs using Ultralytics 8.4.21 on PyTorch 2.8.0 with the framework's default augmentation regime; the full run completed in approximately 15 minutes. Complete training-time hyperparameters are catalogued in Section 3.10.3.

Held-out test mAP@50 reached 0.855 on this detector, clearing the threshold of

0.80 set as success criterion (i) in Section 1.2; full per-class metrics are reported in Chapter 4. One per-class result is relevant here as design evidence: the Player, Goalkeeper, and Referee classes all reach mAP@50 above 0.98 on the test set, but the Ball class achieves only 0.459 with recall 0.393. Inspection of the normalised confusion matrix during development showed that 31% of true balls are classified as background and 40% of predictions assigned to the Ball class are background false positives. This pattern — high precision on the three large-object classes paired with a recall collapse on the ball — is the diagnostic evidence that motivated a separate ball detector.

3.4.2 Dedicated Ball Detector

Two structural factors drive the recall collapse. First, class imbalance: the ball appears once per image on average while players appear roughly twenty times, so localisation loss on player boxes dominates the gradient signal. Second, scale: at broadcast distance the ball occupies only a handful of pixels, far smaller than the typical anchor distribution of single-stage detectors [34, 35]. A single-class detector trained on a ball-curated dataset removes the first issue entirely and lets model capacity focus on the second.

The dedicated detector is a YOLOv8x model trained on the *ball_dataset* (Roboflow), comprising 1,165 frames split 932/114/119 each with exactly one ball annotation. The xlarge variant was chosen because the marginal inference cost is acceptable for a single-class head and a larger backbone typically helps on fine-grained small-object features. Training used the same 1280×1280 resolution and augmentation regime as the generic detector, ran for 150 epochs at batch size 8, and completed in approximately 110 minutes. The resulting model achieves test mAP@50 of 0.950 and recall 0.896 — a 49 percentage-point absolute improvement over the generic detector's ball performance, and the headline diagnostic of Table 3.3.

The dedicated detector is run at inference with a deliberately permissive confidence threshold (`conf`=0.05) so that low-confidence but legitimate ball detections survive into the temporal filtering stage. The filter then selects, from the surviving candidates per frame, the detection nearest to the previous frame's ball centre subject to a maximum displacement of 80 pixels and a minimum confidence of 0.25. The 80-pixel gate is a plausibility constraint on per-frame ball motion at typical broadcast frame rates and resolutions; the 0.25 minimum is the second stage of confidence gating that the permissive detector threshold defers to. The filter relies on the dedicated detector producing a high-recall set of candidates per frame, a behaviour the generic detector

cannot reliably provide.

The generic detector runs at inference with confidence threshold 0.2 and IoU threshold 0.5 for non-maximum suppression, the latter following Equation 2.3 and the procedure of Section 2.4.1. The 0.2 confidence threshold is deliberately low: the downstream tracker performs additional quality gating during association, and recovering a marginal player detection that the tracker can stabilise across frames is preferable to dropping it at the detector stage.

Detector	**P**	**R**	**F1**	**mAP@50**	**mAP@50–95**
Generic (Ball class)	0.818	0.393	0.531	0.459	0.194
Dedicated ball	0.964	0.896	0.929	0.950	0.556

Table 3.3: Ball-class performance on the held-out test set: generic four-class detector versus dedicated single-class detector. The dedicated detector raises ball mAP@50 from 0.459 to 0.950 — a 49 percentage-point absolute improvement that motivates the two-detector architecture. Per-class metrics for all classes of the generic detector are reported in Chapter 4.

3.5 Multi-Object Tracking

The detector outputs of Section 3.4 are stateless, but every downstream tactical output depends on stable identities across frames. The pipeline meets this need with a multi-object tracker, addressing FR2 and the second project objective of Section 1.2.

The tracker follows the tracking-by-detection paradigm of Section 2.5.1: per-frame detections are associated with active tracks by solving a bipartite assignment problem on a cost matrix derived from spatial overlap between predicted and observed boxes. The pipeline invokes this through Ultralytics's unified `model.track()` interface, which wraps the detector call and applies the configured association algorithm in-place. The default tracker is ByteTrack [45], with BoT-SORT [2] available as a drop-in alternative through a configuration flag (`--tracker bytetrack.yaml` or `botsort.yaml`). The two algorithms differ in their association strategies (Sections 2.5.3 and 2.5.4); supporting both lets Chapter 4 compare them like-for-like against success criterion (ii), HOTA ≥ 0.45.

Both trackers run with their Ultralytics-shipped configuration files unmodified, in line with the development methodology of Section 3.3: tracker-internal hyperparameters are well-studied [45, 2] and tuning them was deliberately out of scope. Holding

the configurations to their published defaults also makes the comparative evaluation a fair baseline rather than a hyperparameter sweep.

The tracker emits, per frame, a set of identified detections each carrying a class label, bounding box, confidence score, and stable track identifier. The per-frame loop splits these by class: player and goalkeeper tracks feed the team classifier of Section 3.6, while referee tracks bypass it because referees belong to neither team and are excluded from possession reasoning. The ball is handled outside the multi-object tracker because at most one ball is on the pitch at any moment; the temporal coherence filter of Section 3.4 provides the equivalent single-target stabilisation.

3.6 Team Classification

Producing per-player team labels without manual annotation directly addresses the third project objective in Section 1.2 and FR3 of Table 3.2. The classification problem is genuinely unsupervised: each match brings a novel pair of kits, no labelled training data is available, and the only signal is visual appearance. The pipeline addresses this with the SigLIP–UMAP–KMeans formulation introduced in Section 2.6, integrated through the open-source `sports` library [37], which provides a reference implementation of the embedding extraction, dimensionality reduction, and clustering steps. The contribution of this work lies in the surrounding architecture that makes those building blocks usable inside a per-frame pipeline: a one-shot warmup phase, a torso-focused cropping strategy, track-level temporal smoothing, and a separate goalkeeper resolver. Each is described in turn below.

3.6.1 Warmup-Phase Integration

The clustering algorithm of Equation 2.15 is fitted once per video, in a dedicated warmup phase that runs before the per-frame loop begins. The warmup samples frames at a fixed stride (`warmup_stride`=30, roughly one frame per second at 30 fps) over the first `warmup_seconds`=20 of footage. For each sampled frame the generic detector of Section 3.4 is run, only player-class detections are retained, boxes smaller than 35×35 pixels are dropped to exclude unreliably tiny crops, and the surviving boxes are torso-cropped (Section 3.6.2). The collected crops are capped at `max_warmup_crops`=800 and passed in a single batch to `TeamClassifier.fit()`, which extracts SigLIP embeddings, projects them through UMAP, and fits a $k = 2$ KMeans model.

The decision to fit once rather than incrementally is the methodological core of this design. KMeans is sensitive to the empirical distribution of points seen during fitting, and incremental or per-frame refitting would produce a moving cluster boundary — a player whose embedding sat fractionally on one side of the boundary in frame t could be reassigned in frame $t+1$ purely because the boundary moved, not because the player's appearance changed. Fitting once on a representative twenty-second window stabilises the boundary for the rest of the video. The trade-off is that the system cannot adapt to sudden visual changes such as a substitute appearing in a markedly different kit, but in broadcast football this is benign: kits do not change mid-match, and twenty seconds is sufficient for both teams to appear under typical broadcast direction. The 800-crop cap is an empirical compromise between fit stability and the embedding-extraction latency that dominates warmup time.

3.6.2 Torso Cropping

The crops fed into the embedding model are the upper-body subregions of each player's bounding box, taken between vertical fractions `torso_top`=0.15 and `torso_bottom`=0.65. The motivation is signal-to-noise: the discriminative information for team identity is concentrated in the torso, where the jersey colour, sponsor logo, and crest appear, while the lower body contributes shorts (frequently a colour shared between home and away kits) and grass that varies systematically with the camera angle rather than with team membership. Excluding the top fraction removes the head and a thin halo of background; excluding the bottom fraction removes the lower-body noise just described. Restricting the crop to this band gives the SigLIP encoder a more homogeneous input distribution and reduces the risk of the embedding model learning spurious lower-body or background features as cluster cues.

3.6.3 Track-Level Temporal Smoothing

Per-frame team predictions are noisy. Even with a well-conditioned cluster fit and clean torso crops, individual frames can produce wrong assignments due to motion blur, partial occlusion, an unusual pose, or a player turning their back to the camera in a way that hides the jersey. Yet team identity is a constant property of a track — a player does not change teams during a match. The `TrackMajorityVote` class in `assigner.py` exploits this by maintaining, for each track identifier, a deque of the last `team_smooth`=30 per-frame predictions and emitting the modal value as the smoothed

team label.

Thirty frames is approximately one second at 30 fps. This window is short enough that the smoother reacts quickly when a track identifier is replaced (when the underlying tracker drops a track and a new identifier takes over), but long enough that transient per-frame errors are out-voted by the surrounding correct predictions. The smoothing is applied at the track level rather than per detection, so detections that arrive without a track identifier fall back to the raw per-frame prediction.

3.6.4 Goalkeeper Resolution

The KMeans model is fitted with $k = 2$ because there are two outfield teams. Goalkeeper kits are required by competition rules to be visually distinct from both outfield kits and from each other, so neither of the two fitted clusters reliably corresponds to a goalkeeper, and a goalkeeper's embedding can fall on either side of the cluster boundary essentially by chance. A $k = 3$ fit was considered but rejected: each match contains only two goalkeepers, the warmup window typically captures few goalkeeper crops, and $k = 3$ on a heavily two-team-dominated crop set is poorly conditioned.

The pipeline resolves goalkeeper identity through a separate spatial heuristic implemented in `gk_resolver.py`. For each frame, the bottom-centre image-space coordinate of every outfield player on team 0 is averaged to give a team-0 centroid, and likewise for team 1. Each detected goalkeeper is then assigned to whichever centroid lies nearer in the image plane. The reasoning is positional: goalkeepers spend the overwhelming majority of match time near or behind their own team's most defensive outfield players. Importantly, this heuristic operates in image-space rather than pitch-space, which means it continues to work during frames where the homography of Section 3.7 cannot be updated due to insufficient confident keypoints. The trade-off relies on the implicit assumption that the two teams' centroids project to distinguishable image-space locations, which is valid for the side-on tactical-camera angles that dominate football broadcasts.

Whether the assembled team-classification stack meets success criterion (iii) of Section 1.2 (F1 ≥ 0.85) is reported in Chapter 4.

3.7 Pitch Registration

Pitch registration is the geometric stage that maps image-space detections into pitch coordinates, providing the substrate on which all tactical analytics operate. The pipeline addresses this in two parts: a YOLO-pose model trained to localise a fixed set of pitch landmarks, and a homography estimator that consumes those landmarks via the Direct Linear Transform of Section 2.7.2. This section addresses FR4 and the fourth project objective of Section 1.2.

3.7.1 Keypoint Detector

A YOLOv8x-pose model [15] is fine-tuned to detect a single class (*pitch*) annotated with 32 keypoints corresponding to geometrically distinctive features formed by pitch markings: line intersections, penalty-area and six-yard-box corners, halfway-line endpoints, and tangencies of the centre circle and penalty arcs. The choice of landmark set follows the broadcast-football registration literature [41, 40]; each keypoint has a known position in pitch coordinates, so any subset detected with sufficient confidence yields image-to-pitch correspondences directly usable in DLT.

The detector was trained on the *football_field_detection_dataset* (Roboflow), comprising 317 broadcast frames split 255/34/28 across train/validation/test, using the same A100 hardware and Ultralytics 8.4.21 stack as the detection models of Section 3.4. Training-time hyperparameters are catalogued in Section 3.10.3, but one non-default choice is methodologically substantive enough to surface here: the mosaic augmentation that ships with the YOLO defaults was disabled (`mosaic`=0.0). Mosaic concatenates four training images into a single synthetic input, which is well-motivated for general object detection because it exposes the model to varied object scales and contexts, but the augmentation fragments the global geometric structure of a pitch into four disjoint quadrants. Since the discriminative signal for pitch-keypoint localisation is precisely the spatial relationships between landmarks — a corner flag is identifiable in part because of its position relative to the goal line and the touchline — fragmenting that structure trains the model on inputs whose geometry will never appear at inference. Disabling mosaic was therefore a deliberate departure from defaults motivated by the structural mismatch between the augmentation and the fixed-landmark task.

The trained model achieves test detection mAP@50 of 0.995 (the pitch is, in practice, always present in broadcast frames) and test pose mAP@50 of 0.995 with mAP@50–95 of 0.953. Per-keypoint mean errors cluster around four pixels on the test

set, approximately 0.4% of the image diagonal. One keypoint (kp_16) is an outlier at 18.2 pixels, roughly four times the median; this is reported here for transparency and discussed alongside its downstream effect on homography stability in Chapter 4.

3.7.2 Stride-Cached Homography Estimation

Pitch keypoint inference is the slowest stage of the pipeline by a substantial margin: a separate forward pass through a large pose model, served via a remote inference endpoint. Running it every frame would dominate end-to-end latency. Two observations make this unnecessary. First, broadcast camera motion is slow relative to frame rate — a tactical camera pans over hundreds of milliseconds, not single-frame intervals. Second, the homography itself, once estimated from a stable set of keypoints, is robust to small camera movements. The pipeline exploits this by running keypoint inference only every pitch_stride=15 frames, caching the resulting ViewTransformer, and reusing it for the intervening frames. At 30 fps that recomputes the homography twice per second, well above the rate of meaningful camera motion in broadcast football.

When inference does run, three thresholds gate the construction of a new homography. Each predicted keypoint must clear a per-keypoint confidence threshold of kp_conf=0.5; the resulting set of confident keypoints must contain at least min_good_kps=6 entries; and the inference itself runs with a model-level confidence floor of 0.3. The 6-keypoint minimum is the most consequential of the three. The DLT system in Equation 2.18 requires four non-collinear correspondences for a unique homography, and six provides redundancy that the least-squares solve uses to suppress per-keypoint localisation noise without becoming numerically fragile. If insufficient confident keypoints are present in an inference frame, the previous homography is retained rather than replaced — the camera has not moved significantly in the intervening half-second, so the cached transform remains a reasonable approximation.

The resulting ViewTransformer exposes a vectorised transform_points method that maps any image-space coordinate into pitch-space, applying the projective transformation of Equation 2.16 followed by the dehomogenisation of Equation 2.17. Player anchors and the ball are projected through this method on every frame, supplying the pitch-space coordinates that Section 3.8 consumes. Whether the resulting projections support the qualitatively coherent tactical outputs required by success criterion (iv) is reported in Chapter 4.

3.8 Tactical Reasoning

The four preceding sections supply, on every frame, a set of tracked players with team labels and confident pitch-space correspondences. Tactical reasoning is the stage at which these intermediates become the analytical outputs that motivate the project: bird's-eye trajectories, heatmaps, speed and distance, possession assignments, pass events, and offside flags. All modules in this section operate in pitch-space coordinates, in line with the ground-plane validity argument of Section 2.8.1, and are presented in the order in which they consume each other's outputs. The section addresses FR5 and the fifth project objective of Section 1.2.

3.8.1 Pitch-Space Projection

For each tracked player, the bottom-centre of the bounding box is taken as the ground-plane anchor and projected through the cached homography using the `ViewTransformer` of Section 3.7, following Equation 2.21. The bottom-centre choice is the standard broadcast-analytics convention: it approximates the point where the player's feet meet the playing surface, which is the unique location on the bounding box that lies on the ground plane and therefore satisfies the homography's planar assumption. The ball is projected from its detected centre. All downstream modules consume these pitch-space coordinates, expressed in centimetres with the pitch origin at one corner.

3.8.2 Possession Tracking

Possession is assigned by nearest-eligible-player matching, in the formulation of Equation 2.27: for each frame in which a ball detection is available, the player whose pitch-space anchor lies closest to the ball is identified, and possession is assigned to that player provided the distance is at most `proximity_cm`=350 cm. Referees are excluded from the eligibility set because they should never be in possession; the team-classification stage of Section 3.6 provides this filter through its class labels. If no eligible player lies within the threshold, possession for that frame is left unassigned rather than attributed to the nearest-but-still-distant player. The 350 cm threshold absorbs the combined effect of bounding-box anchor error and homography projection noise, which together can shift a player's estimated pitch position by tens of centimetres even when the underlying detection is correct, while remaining small enough to disambiguate possession in the typical congested-midfield scene.

3.8.3 Pass Detection

Pass detection sits on top of possession assignment. Per-frame possession labels are inherently noisy — a frame in which the ball is in flight between two players can briefly attribute possession to a third player who happens to lie near the ball's trajectory, and a frame in which ball detection fails can break a stable possession run. The pipeline addresses this with a hold-stability requirement: a player is considered to genuinely possess the ball only after the per-frame label has named that player for at least `min_hold_frames`=3 consecutive frames, and a pass is registered only when stable possession transitions from one player to a distinct other. Transitions that fail the stability requirement on either side are discarded as transient noise.

The detection logic is implemented in the `_PossessionTracker` class in `offside.py`. A transition between two players on the same team is logged as a *pass*; a transition between players on opposing teams is logged as an *interception*. Both are written to the JSON event log with the schema:

```
{ "frame_idx": int,
  "passer_track_id": int,
  "passer_team_id": int,
  "ball_pitch_xy": [float, float] }
```

3.8.4 Offside Detection

Offside detection extends the pass-detection module with the geometric criterion of Equation 2.28. The pipeline implements *positional* offside only — the spatial component of Law 11 — and does not attempt to resolve the active-involvement clause, which depends on behavioural cues that broadcast video alone does not reliably expose.

Two pieces of context are required: the attacking direction of each team and the location of the second-last defender at the moment a pass is initiated. The attacking direction is recovered automatically by `_AttackDirectionEstimator`, which accumulates pitch-space x-coordinates of each team over the first `direction_min_samples`=30 sampled frames and assigns the team with the lower mean x as attacking right (and vice versa). The estimator runs once during the early phase of the video and the result is fixed thereafter; mid-half direction switches are out of scope and are recorded as a known limitation in Section 3.11. When the pass-detection module fires, the offside checker locates the second-last defender of the defending team along the attacking x-axis and flags the receiver as offside if their pitch-space x-coordinate is beyond the

defender's by more than `offside_margin_cm`=30 cm. The margin absorbs combined detection-and-projection noise, preventing flags on plays where a player appears fractionally beyond the defender purely because of pixel-level anchor error compounded by homography uncertainty.

Offside events trigger a visual annotation that persists for `display_frames`=50 frames (approximately 1.7 s at 30 fps): the offending player's box is drawn red on the broadcast frame, marked red on the bird's-eye view, and a dashed red line is drawn at the offside x-coordinate to indicate the defender's line. Each event is logged with the schema:

```
{ "frame_idx": int,
  "pass_event": { ... pass-event fields above ... },
  "offside_line_x": float,
  "offside_players": [
    { "track_id": int, "pitch_xy": [float, float] }, ...
  ] }
```

3.8.5 Speed and Distance

Per-track speed and cumulative distance are computed by finite-differencing the pitch-space anchor across consecutive frames, following the velocity formulation of Equation 2.24 and the distance accumulation of Equation 2.26. Speed is reported in km/h and distance in metres in the per-player block of the stats JSON (`distance_m`, `avg_speed_kmh`, `max_speed_kmh`). A plausibility cap (`max_speed_kmh`=40) rejects per-frame displacements implying speeds beyond this threshold: elite sprinters peak around 35–37 km/h, so 40 km/h leaves headroom above the realistic ceiling while still flagging single-frame jumps from tracking and projection noise. Frames whose finite-differenced speed exceeds the cap have their displacement contribution zeroed for that frame, preserving the cumulative distance estimate against single-frame noise spikes at the cost of also rejecting genuine high-speed motion above 40 km/h — a regime that essentially does not occur in football.

3.8.6 Heatmaps

Per-track positional heatmaps are accumulated on a pitch-space grid, following the histogram construction of Equation 2.22. The grid defaults to 480×280 bins covering the full pitch, giving a spatial resolution of roughly 20×25 cm per bin — finer than the

projection-pipeline noise floor and coarser than per-frame jitter, so each bin aggregates many frames of presence rather than acting as a noise multiplier. On each frame the bin containing each player's pitch-space anchor is incremented. At the end of the video, each track's accumulated histogram is post-processed: a Gaussian smoothing pass is applied as in Equation 2.23, followed by a $\gamma = 0.22$ correction to compress dynamic range and reveal less-visited regions. The γ correction is a display choice rather than a statistical operation: the raw histogram is also written to disk as an NPY array so that downstream analysis can operate on the unsmoothed counts.

3.9 Implementation Testing and Verification

The development methodology of Section 3.3 emphasises component-level isolation specifically because errors in any upstream stage propagate through every downstream tactical output. Verifying each component in isolation, on representative input, before relying on it in the integrated pipeline was therefore essential rather than optional.

The verification performed during development is summarised in Table 3.4. None of these procedures constitute formal automated testing in the sense of a continuous integration suite; rather, they are the deliberate inspection steps that the development loop produced as each component was integrated. Quantitative evaluation of the final integrated system against the success criteria of Section 1.2 is reported separately in Chapter 4.

3.10 System Engineering and Reproducibility

3.10.1 Pipeline Orchestration

The modules described in the preceding sections are integrated through a single orchestration script (`run.py`) that exposes the pipeline as a command-line tool. The implementation follows a modular layout under `src/`, with one subpackage per processing stage — `ball/`, `heatmap/`, `pitch/`, `stats/`, `team/`, and `pipeline/` — so that each component can be modified, tested, or replaced independently. The tracker swap is the cleanest instance of this modularity: switching between ByteTrack and BoT-SORT requires only a different configuration filename passed to the Ultralytics tracker interface, with no changes elsewhere in the codebase, satisfying NFR1 of Table 3.2.

Execution proceeds in two phases. The warmup phase of Section 3.6.1 runs once

Component	Verification performed	Purpose
Detection pipeline	Visual inspection of bounding-box overlays on sampled frames; class-filter and confidence-threshold behaviour confirmed against schema.	Confirms downstream modules receive valid, correctly-classed detections.
Ball filter	Spot checks on clips with detection gaps and on clips with rapid ball motion; verified that displacement-gate rejection produced expected behaviour.	Confirms the temporal filter rejects implausible candidates without suppressing legitimate motion.
Team classifier (warmup)	Inspection of the fitted UMAP projection and KMeans cluster boundary on the warmup crop set; confirmed both teams recovered as separated clusters.	Confirms the warmup-phase fit produces a usable two-team boundary before per-frame prediction begins.
Track-level smoothing	Comparison of raw per-frame versus smoothed team labels for a sample of tracks across the development clips.	Confirms smoothing reduces frame-level noise without lagging genuine track-identifier changes.
Homography estimator	Verified that projected pitch coordinates of detected players fall within the pitch bounds for the majority of frames; manual inspection of the bird's-eye view for sanity.	Detects ill-conditioned or failed homography fits that would otherwise corrupt downstream analytics.
Possession module	Frame-level inspection of nearest-player assignments on possession-of-ball sequences from development clips.	Confirms proximity-based assignment behaves as intended in congested midfield scenes.
Pass and offside detection	Manual inspection of detected pass and offside events against the source footage.	Distinguishes genuine passes from possession noise; confirms the offside criterion fires only at pass moments.
Output generation	Verified that all four output formats (annotated MP4, bird's-eye MP4, heatmap PNGs, JSON event log and stats) are produced for each run-mode preset.	Ensures end-to-end pipeline completeness across all modes.

Table 3.4: Component-level verification performed during development. Each row describes a deliberate inspection step performed before the corresponding component was relied on in the integrated pipeline. Quantitative evaluation is reported separately in Chapter 4.

at the start of each video to fit the team-classification model on a representative crop set; the per-frame loop then iterates through the remaining frames in the order shown in Figure 3.1. The two-phase structure is required by the cluster-stability argument of Section 3.6.1 and provides a natural place to log fitting diagnostics separately from per-frame output.

Several caching layers amortise computational cost: pitch keypoint inference is cached every fifteen frames (Section 3.7.2); the previous ball position is held as a one-frame cache for the temporal coherence filter (Section 3.4); per-track team-prediction history is held in a thirty-frame deque (Section 3.6.3); and per-track heatmap arrays are accumulated in memory until end-of-video. The heatmap arrays are the largest cache at 480×280 floats per active track and remain modest in absolute terms, so the pipeline runs in a single Python process without explicit memory management.

3.10.2 Output Formats

The output format reflects the dual audience for the pipeline's results, satisfying NFR3. An annotated MP4 and a bird's-eye-view MP4 are produced for human inspection; per-track heatmap PNGs serve the same purpose. For machine consumption, the raw heatmap counts are also exported as NPY arrays, and detected pass and offside events together with per-player match statistics are serialised to JSON using the schemas of Section 3.8. The JSON outputs are the inputs that Chapter 4 consumes for quantitative assessment of success criterion (iv).

Operational ergonomics are handled by a wrapper shell script (`run.sh`) that exposes mode presets — `preview` for short test clips, `full` for end-to-end processing of complete matches, and `debug` for verbose intermediate logging — so that common invocations do not require remembering the underlying argument set. A separate Streamlit interface provides a graphical front-end accepting a video upload and surfacing the same outputs as the command-line entry point, included as a demonstration that the pipeline can be packaged for use beyond the development environment.

3.10.3 Reproducibility

Three categories of reproducibility commitment apply to the pipeline, satisfying NFR2.

Environment. Training was carried out on an NVIDIA A100-SXM4-80GB GPU using Python 3.11, Ultralytics 8.4.21, PyTorch 2.8.0, and CUDA 12.8; inference is

intended for and developed against an Apple M2 Pro (no discrete GPU). The pipeline runs end-to-end in a single Python process without multi-GPU coordination, simplifying the inference environment.

Models, datasets, and configuration. Three YOLO models are fine-tuned by this project: the four-class generic detector (Section 3.4), the dedicated ball detector (Section 3.4), and the pitch-keypoint detector (Section 3.7). Each is trained on a publicly available Roboflow dataset (*Football_object_detection_dataset*, *ball_dataset*, and *football_field_detection_dataset* respectively), with hyperparameters as reported in the corresponding sections. Trained weights are saved locally; the keypoint detector is additionally served via Roboflow's serverless inference endpoint at runtime. All pipeline-level hyperparameters discussed throughout this chapter are exposed as command-line arguments by `run.py`, with mode-specific defaults in `run.sh`.

Limitations. Two aspects of the pipeline cannot be reproduced bit-exactly: random initialisation in YOLO training was not seeded, so re-running the training procedure will produce numerically distinct (but behaviourally equivalent) weights, and the Roboflow-hosted inference endpoint is a third-party service whose identical availability cannot be guaranteed into the future. Local deployment of the keypoint model on the trained weights is supported as a fallback for replicators without the hosted endpoint.

3.11 Design Decisions and Failure Handling

The previous sections have justified each component-level design decision in the context where the decision was made. To support the marker in seeing the overall design rationale at a glance — and to make the system's known limitations explicit rather than dispersed — this section consolidates both. Table 3.5 lists the principal architectural decisions with their considered alternatives and trade-offs; Table 3.6 records the failure conditions the pipeline is designed to absorb gracefully, the mitigation strategy in each case, and the residual limitation that the mitigation does not fully resolve.

The decisions and limitations recorded above set the scope within which the quantitative evaluation of Chapter 4 should be read: the pipeline is designed to absorb these failure modes gracefully rather than to eliminate them, and the success criteria of Section 1.2 are assessed against this scoped behaviour.

Decision	Alternative	Rationale	Trade-off
Two-detector architecture	Single multi-class detector	49 pp ball-class mAP@50 improvement on the same task (§3.4).	Second inference pass per frame.
Mosaic disabled for keypoint training	Default Ultralytics augmentation	Pitch geometry has fixed structure that mosaic fragments into disjoint quadrants (§3.7).	Fewer augmented training samples; longer training to convergence.
Stride-15 cached homography	Per-frame keypoint inference	Keypoint inference dominates pipeline latency; broadcast camera motion is slow (§3.7).	Brief projection drift during rapid camera moves.
$k = 2$ team clusterer + spatial GK resolver	Three-cluster team model	$k = 3$ on a heavily two-team-dominated crop set is poorly conditioned (§3.6).	Requires a separate goalkeeper-handling pathway.
Fit-once warmup clustering	Per-frame or incremental refit	Cluster boundary stability; avoids label flip-flop from drifting boundary (§3.6.1).	No adaptation to mid-match visual change.
Track-level team smoothing	Per-detection prediction	Team identity is a constant property of a track; absorbs frame-level prediction noise (§3.6.3).	One-second latency in reacting to genuine track-identifier changes.
Positional offside only	Full Law 11 modelling including active involvement	Active involvement depends on behavioural cues that broadcast video does not reliably expose (§3.8.4).	Cannot distinguish active from passive offside positions.
Image-space GK centroid resolution	Pitch-space resolution	Functions before homography is valid; preserves goalkeeper assignment when keypoints are insufficient (§3.6.4).	Implicit assumption of side-on broadcast camera angle.

Table 3.5: Principal architectural decisions, their considered alternatives, and the trade-offs each decision accepts.

Failure condition	Mitigation strategy	Residual limitation
Ball not detected in a frame	Possession is left unassigned for that frame rather than imputed.	Passes may be missed during sustained ball-detection gaps.
Implausible ball position jump	Temporal-coherence filter rejects the candidate; previous ball position is retained.	Genuine high-speed ball motion can be suppressed alongside noise.
Insufficient confident pitch keypoints	Previous cached homography is retained rather than replaced.	Projection drift accumulates during sequences of failed inference.
Track identifier lost or replaced	Track-level smoothing absorbs short label noise; new identifier starts fresh.	Per-track heatmaps and statistics may fragment when tracks split.
Goalkeeper visually similar to an outfield kit	Spatial GK resolver assigns to nearest team centroid in image-space.	Fails for behind-the-goal camera angles.
Half-time attacking-direction switch	Direction estimator runs once early; result is fixed thereafter.	Manual intervention or a rerun is required to handle full-match clips.
Single-frame anomalous speed (e.g. track-id swap)	40 km/h plausibility cap zeroes the displacement contribution for that frame.	Genuine motion above the cap is also rejected, though this regime does not occur in football.

Table 3.6: Failure conditions handled by the pipeline, the corresponding mitigation strategy, and the residual limitation that the mitigation does not fully address. The half-time direction-switch row resolves the forward reference made in Section 3.8.4.

Chapter 4

Evaluation

4.1 Analysis

4.2 Discussion

4.3 Limitations

Chapter 5

Conclusion

5.1 Summary

5.2 Future Work

Bibliography

[1] C. C. Aggarwal, A. Hinneburg, and D. A. Keim. On the surprising behavior of distance metrics in high dimensional space. In *International Conference on Database Theory (ICDT)*, pages 420–434. Springer, 2001.

[2] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky. Bot-sort: Robust associations multi-pedestrian tracking. *arXiv preprint arXiv:2206.14651*, 2022.

[3] M. Beetz, S. Gedikli, J. Bandouch, B. Kirchlechner, N. von Hoyningen-Huene, and A. Perzylo. Visually tracking football games based on tv broadcasts. In *Proceedings of the Twentieth International Joint Conference on Artificial Intelligence (IJCAI)*, 2007.

[4] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016.

[5] A. Cioppa, S. Giancola, A. Deliège, L. Kang, X. Zhou, Z. Cheng, B. Ghanem, and M. Van Droogenbroeck. Soccernet-tracking: Multiple object tracking dataset and benchmark in soccer videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3491–3502, June 2022.

[6] J. Davis, L. Bransen, L. Devos, A. Jaspers, W. Meert, P. Robberechts, J. Van Haaren, and M. Van Roy. Methodology and evaluation in sports analytics: challenges, approaches, and lessons learned. *Machine Learning*, 113:6977–7010, 2024.

[7] A. Deliège, A. Cioppa, S. Giancola, M. J. Seikavandi, J. V. Dueholm, K. Nasber, B. Ghanem, and M. Van Droogenbroeck. SoccerNet-v2: A dataset and benchmarks for holistic understanding of broadcast soccer videos. In *Proceedings of*

the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops, pages 4508–4519, 2021.

[8] P. Dendorfer, A. Osep, A. Milan, K. Schindler, D. Cremers, I. Reid, S. Roth, and L. Leal-Taixe. Motchallenge: A benchmark for single-camera multiple target tracking. *International Journal of Computer Vision*, 129(4):845–881, 2021.

[9] S. Ellens, D. Hodges, S. McCullagh, J. J. Malone, and M. C. Varley. Interchangeability of player movement variables from different athlete tracking systems in professional soccer. *Science and Medicine in Football*, 6(1):1–6, 2022.

[10] J. Fernandez and L. Bornn. Wide open spaces: A statistical technique for measuring space creation in professional soccer. In *MIT Sloan Sports Analytics Conference*, 2018.

[11] F. R. Goes, L. A. Meerhoff, M. J. O. Bueno, D. M. Rodrigues, F. A. Moura, M. S. Brink, M. T. Elferink-Gemser, A. J. Knobbe, S. A. Cunha, R. S. Torres, and K. A. P. M. Lemmink. Unlocking the potential of big data to support tactical performance analysis in professional soccer: A systematic review. *European Journal of Sport Science*, 21(4):481–496, 2021.

[12] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

[13] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition, 2003.

[14] M. Istasse, J. Moreau, and C. De Vleeschouwer. Associating players to teams using visual features trained with weak supervision. In *International Conference on Machine Vision Applications (MVA)*, 2019.

[15] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8, 2023.

[16] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

[17] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 2012.

[18] H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.

[19] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[20] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2117–2125, 2017.

[21] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755. Springer, 2014.

[22] D. Linke, D. Link, and M. Lames. Football-specific validity of TRACAB's optical video tracking systems. *PLOS ONE*, 15(3):e0230179, 2020.

[23] J. Liu, X. Tong, W. Li, T. Wang, Y. Zhang, and H. Wang. Automatic player detection, labeling and tracking in broadcast soccer video. *Pattern Recognition Letters*, 30(2):103–113, 2009.

[24] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg. SSD: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, pages 21–37. Springer, 2016.

[25] L. Lolli, P. Bauer, C. Irving, D. Bonanno, O. Höner, W. Gregson, and V. Di Salvo. Data analytics in the football industry: a survey investigating operational frameworks and practices in professional clubs and national federations from around the world. *Science and Medicine in Football*, 2024.

[26] J. Luiten, A. Osep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe. A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision*, 129(2):548–578, 2021.

[27] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297, 1967.

[28] F. Magera, T. Hoyoux, O. Barnich, and M. Van Droogenbroeck. BroadTrack: Broadcast camera tracking for soccer. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, Tucson, Arizona, USA, February 2025.

[29] A. Maglo, A. Orcesi, J. Denize, and Q. C. Pham. Individual locating of soccer players from a single moving view. *Sensors*, 23(18):7938, 2023.

[30] L. McInnes, J. Healy, N. Saul, and L. Großberger. Umap: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29):861, 2018.

[31] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024.

[32] K. O'Shea and R. Nash. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.

[33] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.

[34] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.

[35] J. Redmon and A. Farhadi. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.

[36] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[37] Roboflow. sports: A python library for sports analysis, 2024. GitHub repository, accessed 2025–2026.

[38] V. Somers, V. Joos, A. Cioppa, S. Giancola, S. A. Ghasemzadeh, F. Magera, B. Standaert, A. M. Mansourian, X. Zhou, S. Kasaei, B. Ghanem, A. Alahi, M. Van Droogenbroeck, and C. De Vleeschouwer. Soccernet game state reconstruction: End-to-end athlete tracking and identification on a minimap. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3293–3305, June 2024.

[39] The International Football Association Board. Laws of the game 2025/26. https://downloads.theifab.com/downloads/laws-of-the-game-2025-26-single-pages?l=en, 2025. Accessed 2026-04-08.

[40] J. Theiner and R. Ewerth. Tvcalib: Camera calibration for sports field registration in soccer. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1166–1175, 2023.

[41] J. Theiner, W. Gritz, E. Müller-Budack, R. Rein, D. Memmert, and R. Ewerth. Extraction of positional player data from broadcast soccer videos. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1463–1473, 2022.

[42] F. Vidal-Codina, N. Evans, B. El Fakir, and J. Billingham. Automatic event detection in football using tracking data. *Sports Engineering*, 25(1), 2022.

[43] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems*, volume 27, 2014.

[44] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. *arXiv preprint arXiv:2303.15343*, 2023.

[45] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. Bytetrack: Multi-object tracking by associating every detection box. In *Computer Vision – ECCV 2022*, volume 13682 of *Lecture Notes in Computer Science*, pages 1–21. Springer, 2022.

Appendix A

Additional Materials